AI BASED EMERGENCY CALL AND SMS TRIAGE SYSTEM FOR DISASTER RESPONSE IN SRI LANKAN LOW RESOURCE LOCAL LANGUAGES

INDIVIDUAL THESIS

Amandhi Navoda Siriwardena

IT22504418

Individual component: Multi-Source Incident Fusion: Deciding Whether Multilingual Emergency Calls and SMS Reports Describe the Same Incident

Group ID: R26_DS_011

Report submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Specialising in Data Science

Department of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Amandhi Navoda Siriwardena

Student ID: IT22504418

Signature:  Date: 10/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Dr. Prasanna Sumathipala		
Co-supervisor: Ms. Malithi Nawarathne		

ABSTRACT

Emergency services in Sri Lanka receive reports of floods and fires by voice call and SMS, in Sinhala, Tamil, English and code-mixed forms. One emergency is often reported several times, while different emergencies can share a road, a type and a time. Merging two real emergencies can leave one without a response; recording one emergency as several wastes scarce resources. Existing duplicate-call handling relies on geocoded addresses, a single language and the call-taker's judgement. This thesis presents the multi-source incident fusion component of a four-component intake pipeline. For each incoming report it reconciles three extraction sources while keeping the caller's words verbatim, scores the report against every open incident on four independently learned channels (cross-script place comparison, per-type temporal decay, a fine-tuned LaBSE encoder and a fine-tuned mDeBERTa evidence judge that separates contradictions from count differences), fuses them into a calibrated probability with gradient boosting, and merges only when the probability exceeds a threshold derived from a 3:1 cost ratio of false merges to false splits. Uncertain and ambiguous cases are flagged for review, and every merge carries an evidence trail. All choices were made on a development split with adoption rules written in advance; the test split was reported once. On a purpose-built corpus of 535 reports, the live configuration wrongly merged 0.9% of test reports (merge precision 98.1%) and linked 70.7% of repeat reports, or 84.0% at 1.7% false merges without a conservative no-location rule. Matching against whole incidents instead of single reports cut false splits from 24.0% to 10.7%. Google Places placed only 53-58% of native-script locations within 1 km, against 83% for Latin script, which motivated learned text comparison. The component runs on a CPU and is integrated live with the other components and a dashboard.

Keywords: incident fusion, multilingual emergency reports, evidence matching, code-mixed Sinhala and Tamil, cost-sensitive decision

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Dr. Prasanna Sumathipala, and my co-supervisor, Ms. Malithi Nawarathne, for their guidance, encouragement and constructive feedback throughout this research. Their advice on the design of the evaluation, in particular on reporting results honestly, shaped this work.

I thank the members of my research group, who developed the urgency detection, clarification dialogue and SMS triage components, for their cooperation in agreeing the shared incident schema and the integration contract, for the changes they made on their side to support the fusion component, and for the many live test sessions in which we ran the four components together. I also thank the staff of the Faculty of Computing of the Sri Lanka Institute of Information Technology for the facilities and support provided during the project.

Finally, I am grateful to my family for their patience and support during this research.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall Project Context 1
1.2 Individual Component Context 2
1.3 Problem Statement 3
1.4 Research Gap 4
1.5 Aim and Objectives 5
1.6 Research Questions or Hypotheses 6
1.7 Scope Limitations and Assumptions 6
1.8 Individual Contribution 7
1.9 Thesis Structure 8
2 LITERATURE REVIEW AND RELATED WORK 9
2.1 Review Method and Scope 9
2.2 Domain and Theoretical Foundations 9
2.2.1 Emergency reporting and dispatch 9
2.2.2 Probabilistic record linkage 10
2.2.3 Probability calibration and cost-sensitive decisions 10
2.3 Related Research 11
2.3.1 Event coreference and topic detection 11
2.3.2 Crisis informatics 11
2.3.3 Multilingual and code-mixed language processing 12
2.3.4 Natural language inference and evidence matching 12
2.3.5 Knowledge distillation and counterfactual data 13
2.4 Existing Systems Products and Techniques 13
2.5 Gap Synthesis and Conceptual Framework 14

3 RESEARCH METHODOLOGY 16
3.1 Research Design 16
3.2 Data Participants or Study Materials 17
3.2.1 Source corpus 17
3.2.2 Report pairs and trap cases 17
3.2.3 Reference labels and human agreement 18
3.2.4 External resources 19
3.3 Experimental or Study Procedure 19
3.4 Measures and Evaluation Metrics 20
3.5 Data Analysis 21
3.6 Ethics Privacy Security and Responsible AI 21
3.7 Reproducibility and Research Artefacts 22
4 PRODUCT AND SOLUTION DESIGN 23
4.1 Requirements 23
4.2 Component Architecture within the Complete Solution 24
4.3 Design Alternatives and Decisions 24
4.4 Detailed Design 25
4.4.1 Step 1: input understanding 25
4.4.2 Step 2: evidence pool 26
4.4.3 Step 3: geography 27
4.4.4 Step 4: time and the candidate cap 27
4.4.5 Step 5: semantic similarity and evidence matching 28
4.4.6 Steps 6 and 7: fusion and decision 29
4.4.7 Steps 8 and 9: incident record and revision 30
4.5 Standards and Professional Practice 30
5 IMPLEMENTATION INTEGRATION AND TESTING 32

5.1 Development Environment and Technology Stack 32
5.2 Component Implementation 32
5.2.1 Processing pipeline 32
5.2.2 Live service 33
5.2.3 Operator page and incident dashboard 33
5.3 Integration with the Other Components 35
5.4 Verification and Software Testing 36
5.5 Deployment and Operation 37
6 RESULTS AND VALIDATION 39
6.1 Research Results 39
6.1.1 Step 1: extraction and reconciliation 39
6.1.2 Step 2: evidence pool 40
6.1.3 Step 3: geography 41
6.1.4 Step 4: time 42
6.1.5 Step 5A: semantic similarity 43
6.1.6 Step 5B: evidence judge 44
6.1.7 Step 6: fusion 46
6.1.8 Step 7: cost-based decision 47
6.1.9 Step 8: incident-level matching 48
6.1.10 Step 9: revision and the ambiguity flag 49
6.2 Product and Technical Results 50
6.3 User or Stakeholder Validation 51
6.4 Integration Results 51
6.5 Summary of Findings 52
7 DISCUSSION 53
7.1 Interpretation of Findings 53

7.2 Comparison with Literature Baselines and Existing Solutions 54
7.3 Research Contribution 55
7.4 Product Contribution 55
7.5 Limitations and Threats to Validity 55
7.6 Implications 57
8 COMMERCIALISATION AND SUSTAINABILITY 58
8.1 Value Proposition and Target Users 58
8.2 Market and Competitive Position 58
8.3 Business or Sustainability Model 59
8.4 Scalability Intellectual Property and Compliance 59
8.5 Contribution to the Sustainable Development Goals 60
9 CONCLUSION AND FUTURE DIRECTIONS 61
9.1 Conclusion 61
9.2 Achievement of Objectives 61
9.3 Individual Research and Product Contributions 62
9.4 Future Directions 62
REFERENCES 64
GLOSSARY 67
APPENDICES 68
Appendix A Extended Results 68
Appendix B Academic Integrity and AI Use Disclosure 69

LIST OF FIGURES

Figure 1.1: Architecture of the group project: C2 and C3 take reports from callers, C1 adds urgency to calls, and C4 fuses the reports of all channels into incidents 2

Figure 4.1: The nine processing steps of the fusion component (C4) 24

Figure 4.2: Learned probability that a report belongs to the same incident as a function of the time gap, per incident type 28

Figure 5.1: Operator testing page: live reports from C2 and C3 (left) and the output of each processing step for the selected report (right) 34

Figure 5.2: Incident dashboard with key figures, incident map and channel overlap 35

Figure 5.3: Integration of the fusion component with C1, C2 and C3 36

Figure 6.1: Share of gold locations that Google Places resolved to within 1 km of the true place, by script 41

Figure 6.2: Step 3B TEST AUC by kind of report pair 42

Figure 6.3: Recall of true links as a function of the candidate cap K on the stress replay 43

Figure 6.4: AUC of semantic similarity on the report pairs: baselines and the fine-tuned encoder 43

Figure 6.5: TEST AUC of semantic similarity by language of the report pair 44

Figure 6.6: TEST macro-F1 of the evidence judge against baselines (four labels) ... 45

Figure 6.7: Reliability diagram of the fusion probability 46

Figure 6.8: Channel ablation of the fusion model (TEST log loss when each channel is removed) 47

Figure 6.9: Error rates as a function of the cost ratio (DEV replay, report-level matching, augmented fusion model) 48

Figure 6.10: Report-level against incident-level matching on TEST 49

Figure 6.11: Final operational results: DEV (mean of ten fold divisions) and TEST 50

LIST OF TABLES

Table 2.1: Comparison of related approaches with the requirements of multilingual emergency report fusion 14

Table 3.1: Size of the C4 dataset (final version) 17

Table 3.2: Dataset versions and the reason for each change 19

Table 3.3: Evaluation metrics used for each part of the component 20

Table 4.1: Functional requirements of the fusion component 23

Table 4.2: Non-functional requirements of the fusion component 23

Table 4.3: Main design alternatives and the option chosen 25

Table 5.1: Technology stack of the fusion component 32

Table 5.2: Examples of faults found in live testing and how they were handled 37

Table 6.1: Accuracy of the reconciled Step 1 fields against the gold fields (% of scorable reports) 39

Table 6.2: Step 2 and Step 1c readers on TEST: learned models against rules 41

Table 6.3: Adoption rule for the third evidence judge run (all values on DEV) 45

Table 6.4: Revision variants on the DEV replay 49

Table 6.5: Final results of the live configuration (DEV: mean (sd) over ten fold divisions; TEST: once) 50

Table 9.1: Achievement of the objectives 61

Table A.1: Step 8 DEV replay: report-level against incident-level matching (single fold division) 68

Table A.2: Multi-seed re-check of the fusion decisions (DEV, mean (sd) over ten fold divisions) 68

Table A.3: Endpoints of the C4 service 68

Table B.1: Generative AI used within the research method 69

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
AUC	Area Under the ROC Curve
B^3 (B-cubed)	Clustering precision and recall averaged over items
C1-C4	Components 1 to 4 of the group project
CAD	Computer-Aided Dispatch
CPU	Central Processing Unit
DEV	Development (validation) split
DMC	Disaster Management Centre (Sri Lanka)
ECE	Expected Calibration Error
FM	False Merge
FS	False Split
GPU	Graphics Processing Unit
IAST	International Alphabet of Sanskrit Transliteration
JSON	JavaScript Object Notation
LaBSE	Language-agnostic BERT Sentence Embedding
LLM	Large Language Model
LR	Logistic Regression
MPDS	Medical Priority Dispatch System
mT5	Multilingual Text-to-Text Transfer Transformer
NLI	Natural Language Inference
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SDG	Sustainable Development Goal
SMS	Short Message Service
TDT	Topic Detection and Tracking
TF-IDF	Term Frequency-Inverse Document Frequency

1 INTRODUCTION

1.1 Overall Project Context

Emergency services in Sri Lanka receive reports of floods, fires and other hazards through two main channels: voice calls to hotlines and short text messages (SMS). Callers speak Sinhala, Tamil or English, and very often mix them. A caller in Colombo may begin in Sinhala, name a road in English and give a count in Singlish (Sinhala written in Latin letters); a caller in the North or East may switch between Tamil and English in the same way (Tanglish). People who report an emergency are frightened, short of time and frequently unsure of details such as the number of people trapped. Disaster events such as the monsoon floods that affect the Western, Southern and Sabaragamuwa provinces also produce sudden peaks in report volume, so a dispatch centre must make many decisions quickly and with incomplete information.

The group project addresses this setting with an integrated, four-component intake and triage pipeline for multilingual emergency reports. Each component takes responsibility for one part of the problem and all four share a common incident schema, so that the output of one component can be consumed by another without manual re-entry. Figure 1.1 shows the overall architecture.

- Component 1 (C1), urgency detection, classifies the urgency of a voice call as High, Medium or Low by combining acoustic features of the caller's speech with linguistic and contextual features of the transcript.
- Component 2 (C2), the adaptive clarification dialogue, conducts the spoken conversation with a caller. It asks the single most useful missing question at each turn, in the caller's own language, and records the conversation turn by turn together with the slots it has filled.
- Component 3 (C3), SMS triage, holds a text conversation with an SMS reporter, extracts structured incident data from noisy, code-mixed messages, and assigns a priority code with the Medical Priority Dispatch System (MPDS).
- Component 4 (C4), multi-source incident fusion, is the subject of this thesis. It receives every finished call from C2 (with the C1 urgency verdict attached) and

every finished SMS conversation from C3, and decides whether each new report describes an incident that is already open or a new incident.

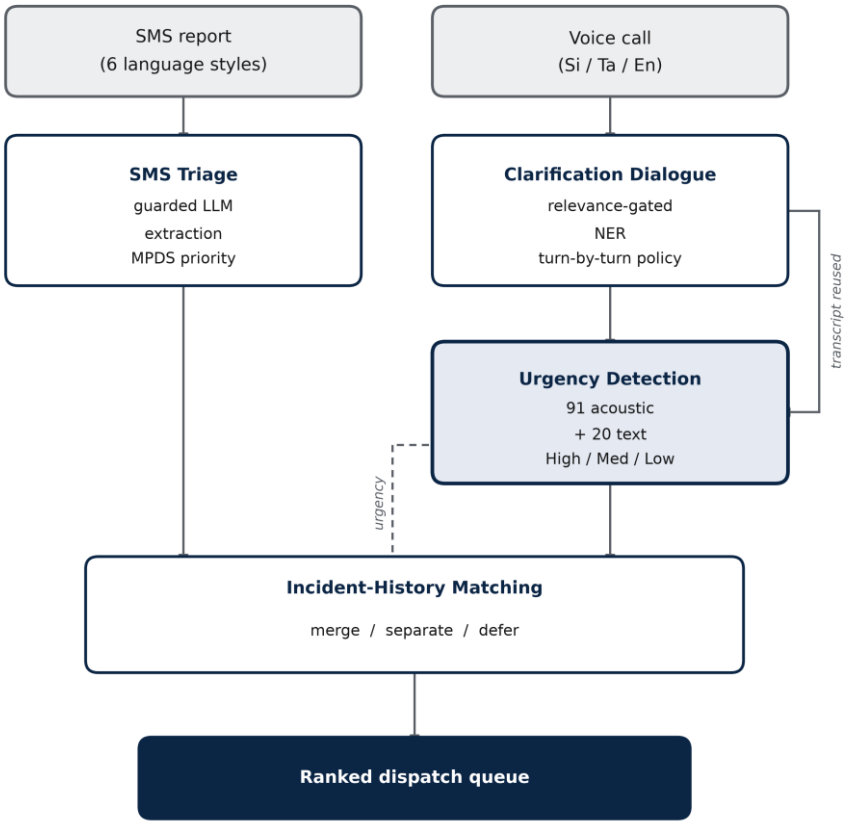


Figure 1.1: Architecture of the group project: C2 and C3 take reports from callers, C1 adds urgency to calls, and C4 fuses the reports of all channels into incidents

The first three components each improve the quality of a single report. None of them, however, can see that a call received at 10:35 and an SMS received at 10:37 describe the same flooded house, or that two calls naming the same street describe two different houses that need two separate rescue teams. That question belongs to C4.

1.2 Individual Component Context

When a disaster affects a populated area, several people often report the same event. A family trapped by rising water may call the hotline, a neighbour may send an SMS, and a relative may call again later to ask whether help has been sent. If a dispatch centre treats each report as a new incident, rescue resources are duplicated and the operator's picture of the situation becomes cluttered. If, on the other hand, two genuinely different emergencies are merged into one, one of them may receive no

response at all. The second error is far more serious than the first, because a duplicated dispatch wastes effort while a missed dispatch can cost lives.

C4 is the component that makes this decision. For every incoming report it answers one question: does this report belong to an incident that is already open (MERGE), or is it a new incident (SEPARATE)? To answer it, C4 compares the incoming report with the open incidents on four independent kinds of evidence: where the event is (geography), when it was reported (time), what the caller said overall (semantic similarity), and whether the individual facts the caller gave agree or conflict with the facts already known about the incident (evidence matching). It then fuses this evidence into a calibrated probability that the report and the incident are the same, and turns that probability into a decision using the asymmetric cost of the two possible errors. Finally, it maintains a fused incident record that shows a dispatcher what is known about each incident, which report said what, and why each merge was made.

The component works on the reports exactly as the callers produced them. The caller's words are never translated or rewritten, because a translation can silently remove the very detail that separates two incidents, such as a house number written as a Sinhala number word. C4 also treats the upstream components' output with appropriate caution: the values that C2 and C3 send are stored as sent and compared with C4's own reading of the conversation, but they are not assumed to be correct.

1.3 Problem Statement

The problem addressed by this component can be stated as follows. Given a stream of emergency reports that arrive one at a time over voice and SMS, in Sinhala, Tamil, English and their code-mixed forms, decide for each report, at the moment it arrives, whether it describes an incident that is already open or a new incident, in a way that (a) keeps the rate of wrongly merged distinct emergencies as low as possible, (b) still links most repeat reports of the same emergency, (c) gives a calibrated probability rather than an unexplained label, and (d) leaves an auditable trail that a human operator can inspect and correct.

Several properties of the setting make this difficult. First, the same place can be named in different scripts and spellings ('14 pansala para gampaha' and '14 පන්සල පාර ගම්පල' are the same address), while two different places can differ by a single digit ('14' and '23' on the same road). Second, many reports contain no usable location at all, for example 'අපේ පැත්තට' ('to our side'). Third, counts are unreliable: one caller reports ten people and another reports twelve for the same house, which is normal variance between witnesses rather than evidence of a different incident. Fourth, the structured fields provided by the upstream components are sometimes wrong, for example when an operator's menu option is recorded as the caller's statement. Finally, the decision must be made within the time budget of the calling component, which allows roughly twenty seconds for the whole round trip.

1.4 Research Gap

Grouping reports that describe the same real-world event has been studied under several names: record linkage and duplicate detection in databases [1], [2], [3], event coreference in news text [4], [5], topic detection and tracking [6], and the filtering and clustering of social media messages during crises [7], [8], [9]. Chapter 2 reviews this work in detail. Four gaps stand out when it is applied to multilingual emergency reports in Sri Lanka.

- Language and script. Most event coreference and crisis informatics resources are in English or in high-resource languages. Sinhala and Tamil remain low-resource languages for natural language processing [10], and the romanised, code-mixed forms that callers actually use (Singlish and Tanglish) are rarely covered by existing datasets or models [11].
- Granularity of conflict. Most similarity-based approaches reward shared words. In emergency dispatch the decisive evidence is often a small conflict inside otherwise identical text: the same road with a different house number, or the same school in a different town. A system must recognise contradiction, not only similarity.
- Asymmetric cost. Record linkage and coreference are usually evaluated with symmetric measures. In dispatch, merging two different emergencies is far more

costly than splitting one emergency into two records, and the decision rule should reflect this explicitly.

- Accumulated history. Most pairwise approaches compare the new report with one earlier report at a time. Evidence about a real incident is spread across several reports, and a fair comparison should be against everything already known about the incident.

To the best of the author's knowledge, no published system combines cross-script place comparison, evidence-level contradiction detection, per-incident-type temporal modelling and cost-sensitive decisions for multilingual emergency reports from voice and SMS channels. This component is an attempt to fill that gap.

1.5 Aim and Objectives

The aim of this component is to design, build and evaluate a multilingual, multi-source incident fusion component that decides, report by report, whether a new emergency call or SMS belongs to an open incident, with a calibrated probability, a cost-aware decision and an auditable evidence trail. The specific objectives are the following.

- O1. To turn each incoming report from C2 or C3 into a reliable structured record by reconciling three sources (the upstream fields, question-anchored recovery from the conversation, and a learned extractor) without translating or altering the caller's words.
- O2. To build independent, learned scoring channels for geography, time, overall meaning and fine-grained evidence, each of which works across Sinhala, Tamil, English and their code-mixed forms.
- O3. To fuse these channels into a calibrated probability that a report and an incident are the same, and to show through ablation which channels contribute.
- O4. To convert the probability into a MERGE or SEPARATE decision using the asymmetric cost of false merges and false splits, and to flag uncertain cases for human review instead of acting on them silently.

- O5. To evaluate the component honestly on held-out data, choosing every setting on a development split and reporting the test split once, and to integrate it live with C1, C2 and C3 together with an operator dashboard.

1.6 Research Questions or Hypotheses

The work is organised around the following research questions.

- RQ1. Can learned models compare place descriptions across Sinhala, Tamil and Latin script well enough to support incident linking when geocoding is unreliable or impossible?
- RQ2. Does an evidence-level judge that labels pairs of caller statements as corroborating, contradicting, neutral or differing only in a count add information beyond overall semantic similarity?
- RQ3. Does comparing a new report against the accumulated evidence of an incident, rather than against single earlier reports, reduce false splits without increasing false merges?
- RQ4. What trade-off between false merges, false splits and review load does a cost-based decision threshold give, and how stable are these numbers across data splits?

The central hypothesis is that a late fusion of independently trained, multilingual channels, combined with incident-level matching and a cost-derived threshold, can keep false merges close to one percent of reports while linking the large majority of repeat reports.

1.7 Scope Limitations and Assumptions

The component is limited to the incident types handled by the group project, mainly floods and fires, and to reports that reach it from C2 and C3 through the agreed contract. It does not perform speech recognition (C2 supplies the transcript) and it does not decide urgency or priority: the C1 urgency verdict and the MPDS code are carried through to the dispatcher for display only and are never used as an input to the merge decision. This separation is deliberate, because an urgent report is not more or less likely to be a duplicate, and letting urgency influence the merge decision would hide errors behind a plausible-looking signal.

The main assumptions are the following. Reports are processed in arrival order. Each report carries the full conversation, with operator and caller turns separated. The evaluation corpus is a purpose-built simulated corpus of emergency dialogues rather than real hotline traffic; real recordings were available to the group only for the urgency component. Results should therefore be read as estimates under controlled conditions rather than as guarantees of field performance, and this limitation is discussed in Chapter 7.

1.8 Individual Contribution

The author designed, implemented and evaluated the complete fusion component (C4) and its integration with the other three components. The individual contribution comprises the following.

- The nine-step design of the component (input understanding, evidence pool, geography, time, semantic similarity, evidence matching, fusion, decision, incident record and revision) and its data contract with C2 and C3.
- Manual creation of the simulated dialogue corpus and its preparation as the C4 training-ready dataset: incident-level splits, trap groups of confusable incidents, labelled report pairs, the evidence pairs for the judge model, and a series of versioned corrections, each built from the previous version without editing it in place.
- Training and verification of the learned models: a distilled mT5-small extractor, readers for question type, answer type and evidence category, a cross-script place model, temporal models per incident type, a fine-tuned LaBSE encoder, a fine-tuned mDeBERTa evidence judge, and the incident-level fusion model.
- The evaluation protocol: development-only selection with adoption rules written before each run, single use of the test split, multi-seed checks, and replay of the live code path in arrival order.
- The live service, its integration with C1, C2 and C3, the operator testing page and the incident dashboard.

The other components (C1, C2 and C3) were developed by the other members of the group. Changes on their side that were needed for integration, such as sending the full turn history, are described in Section 5.3.

1.9 Thesis Structure

Chapter 2 reviews the literature on record linkage, event coreference, crisis informatics, multilingual representation learning and natural language inference, and synthesises the gap. Chapter 3 describes the research design, the dataset, the experimental procedure and the evaluation measures. Chapter 4 presents the requirements and the design of the component. Chapter 5 describes the implementation, the integration with the other components and the testing. Chapter 6 reports the results, and Chapter 7 discusses them, including limitations and threats to validity. Chapter 8 considers commercialisation and sustainability, and Chapter 9 concludes and outlines future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review was organised around the decisions that the component must make rather than around a single research field. For each decision (are two records the same entity, are two place names the same place, do two statements agree, how should a probability become an action) the most relevant line of research was identified and its foundational and recent work was read. Sources were found through Google Scholar, the ACL Anthology, the ACM Digital Library and IEEE Xplore, using combinations of terms such as record linkage, entity resolution, duplicate detection, event coreference, topic detection and tracking, crisis informatics, multilingual sentence embeddings, cross-lingual natural language inference, knowledge distillation, probability calibration and cost-sensitive learning. Work on Sinhala and Tamil language processing and on code-mixed text was searched for separately because it is small and scattered.

Priority was given to peer-reviewed work, to the original papers of methods that the component uses, and to surveys that summarise a field. Commercial dispatch products were reviewed from publicly available descriptions only, because their internal methods are not published. The review covers the period from the classical probabilistic record linkage model of 1969 to multilingual transformer models published up to 2023.

2.2 Domain and Theoretical Foundations

2.2.1 Emergency reporting and dispatch

An emergency dispatch centre receives reports, records them as incidents and assigns resources to incidents. Structured protocols such as the Medical Priority Dispatch System (MPDS) [12] standardise the questions that a call-taker asks and assign a priority code from the answers. Protocols of this kind decide how urgent a single report is; they do not decide whether two reports describe the same incident, which is left to the judgement of the call-taker or to simple rules in the dispatch software, such as flagging calls from nearby addresses within a short time window. In a large-scale

disaster, when hundreds of reports arrive within hours, this judgement becomes a bottleneck.

Two kinds of error are possible. A false merge attaches a report to an incident it does not belong to; the second emergency may then receive no response. A false split records a repeat report as a new incident; this duplicates effort and clutters the picture, but the emergency is still visible to the operator. In decision theory these errors have different costs, and the optimal decision rule depends on their ratio [13]. This asymmetry is a defining property of the dispatch setting and shapes the design in Chapter 4.

2.2.2 Probabilistic record linkage

Fellegi and Sunter [1] formalised record linkage as a decision problem over pairs of records. Each pair is compared on several fields, the agreement pattern is scored with a likelihood ratio between matching and non-matching pairs, and two thresholds divide the pairs into links, non-links and possible links that are sent for clerical review. Three ideas from this model carry directly into the present component: comparison is done field by field, the evidence from different fields is combined into a single score, and uncertain cases are sent to a human instead of being decided automatically. Later surveys of duplicate detection [2] and data matching [3] describe the many field comparison functions that have been proposed, such as edit distances and phonetic codes, and the use of blocking to avoid comparing every record with every other record. The candidate cap described in Section 4.4.4 plays the role of blocking.

2.2.3 Probability calibration and cost-sensitive decisions

A probability is useful for a cost-based decision only if it is calibrated, that is, if events predicted with probability 0.8 occur about 80% of the time. Platt scaling [14] fits a logistic function to the scores of a classifier on held-out data, and isotonic regression offers a non-parametric alternative [15]. Guo et al. [16] showed that modern neural networks are often over-confident and that simple post-hoc calibration corrects much of this. Calibration is measured with the Brier score [17], log loss and the expected calibration error. Elkan [13] showed that, for a calibrated probability p and misclassification costs C_FM for a false merge and C_FS for a false split, the

cost-minimising rule is to merge when p exceeds C_FM / (C_FM + C_FS). The component uses exactly this rule (Section 4.4.6).

2.3 Related Research

2.3.1 Event coreference and topic detection

Cross-document event coreference asks whether two mentions in different documents refer to the same real-world event. Datasets such as ECB+ [4] were built to make the task harder by including documents about similar but different events, for example two earthquakes, so that lexical overlap alone is not enough. Neural coreference models [5] score pairs of mentions with learned span representations. The observation that matters most for this work is that the hard cases are near-duplicates that differ in one decisive detail. The C4 dataset follows the same principle: it contains trap groups of incidents that share a place, a type or a day but are different incidents (Section 3.2).

Topic detection and tracking (TDT) [6] studied the detection of new events in a stream of news stories and the tracking of stories about known events. TDT introduced the on-line setting in which each story must be assigned when it arrives, without knowledge of future stories, which is also the setting of the present component. TDT systems compared a new story with clusters of earlier stories rather than with single stories, an idea that corresponds to the incident-level matching of Section 4.4.7.

2.3.2 Crisis informatics

Research on social media during disasters has shown that affected people report situational information such as damage, needs and locations, and that this information is fragmented and repetitive [9]. Imran et al. [7] surveyed methods for processing such messages, including classification into information types, extraction of locations and clustering of messages about the same event. CrisisLex [8] and human-annotated crisis corpora [18] provide lexicons and labelled data, mostly in English and other high-resource languages. These methods work on public posts aimed at a broad audience, while emergency calls and SMS reports are private conversations guided by an operator's questions. The question-and-answer structure

is an advantage that the present component exploits: an answer to 'where are you?' is a place description even when it contains no recognisable place word.

2.3.3 Multilingual and code-mixed language processing

Multilingual transformer encoders trained on many languages, such as XLM-R [19], made it possible to transfer models across languages. For sentence-level similarity, Sentence-BERT [20] trained siamese encoders to produce embeddings whose cosine similarity reflects meaning, and Reimers and Gurevych [21] extended such embeddings to many languages by knowledge distillation. LaBSE [22] is a language-agnostic sentence encoder trained on translation pairs in more than one hundred languages, including Sinhala and Tamil, and is designed so that translations of the same sentence receive similar embeddings. This property is exactly what is needed when one caller speaks Sinhala and another types English about the same event.

Sinhala and Tamil are nevertheless low-resource languages for language processing. A survey of Sinhala tools and resources [10] found few publicly available datasets and models. Code-mixed text adds a further difficulty: romanised Sinhala and Tamil have no standard spelling, and the same word can be written in many ways. Work on code-mixed Tamil-English text [11] has shown that models trained on native-script text perform poorly on romanised, mixed input. For this reason the component fine-tunes its encoders on its own multilingual data and measures performance separately for Singlish and Tanglish (Section 6.1).

2.3.4 Natural language inference and evidence matching

Natural language inference (NLI) classifies a pair of sentences as entailment, contradiction or neutral. The SNLI corpus [23] made large-scale training possible in English, and XNLI [24] extended evaluation to fifteen languages, which, however, do not include Sinhala or Tamil. DeBERTa [25] improved transformer pre-training with disentangled attention, and its multilingual version mDeBERTa-v3 is among the strongest cross-lingual encoders of its size. The evidence judge in this component (Section 4.4.5) adapts NLI to the emergency setting with four labels: corroborates, contradicts, neutral and count_variance. The last label is needed because two witnesses often give different counts for the same incident; in the standard NLI

scheme such a pair would be a contradiction, but in dispatch it is not evidence of a different incident.

2.3.5 Knowledge distillation and counterfactual data

Large language models extract structured information from text well but are costly, require network access and can produce values that are not supported by the input. Knowledge distillation [26] trains a small student model to reproduce the outputs of a larger teacher. The text-to-text model mT5 [27] is a natural student for multilingual extraction, because it can generate structured output in any of its pre-training languages. The component uses a guarded large language model as a teacher and a fine-tuned mT5-small as the runtime extractor (Section 4.4.1).

Kaushik et al. [28] showed that counterfactually augmented data, in which a human makes the minimal change to an example that flips its label, helps models learn the features that actually matter rather than spurious correlations. The component uses the same idea twice: synthetic copies of real reports that differ only in the house number, labelled as different incidents, and copies of real same-incident pairs whose type features are flipped, labelled as different (Sections 4.4.6 and 6.1).

2.4 Existing Systems Products and Techniques

Commercial computer-aided dispatch (CAD) systems used by emergency services record calls as incidents and usually offer some form of duplicate-call handling, for example by showing the call-taker other open incidents near the caller's address or by matching the caller's telephone number. These features assume a geocoded address, a single language and a human who makes the final decision. They do not compare the content of what different callers said, and they are not designed for code-mixed Sinhala and Tamil. In Sri Lanka, emergency and disaster reports are received by national hotlines such as the Disaster Management Centre's 117 line, where call-takers record reports manually.

Geocoding services such as the Google Places API [29] turn a place description into coordinates. They work well for well-formed Latin-script addresses but less well for informal descriptions and native-script queries; the measurement in Section 6.1 shows that, on the C4 corpus, Places resolved Latin-script locations to within 1 km of

the true place for 83% of reports but Sinhala-script and Tamil-script locations for only 53% and 58%. Open gazetteers such as GeoNames [30] list place names with coordinates and alternate names, but native-script names are available for only a small fraction of Sri Lankan places.

Table 2.1 summarises how the main lines of related work compare with the requirements of the present problem.

Table 2.1: Comparison of related approaches with the requirements of multilingual emergency report fusion

Approach	Multilingual / code-mixed	Detects conflicting detail	Cost-aware decision	Uses accumulated history	Calibrated output
Probabilistic record linkage [1]	No	Field level only	Two thresholds	No	Partly
Event coreference (ECB+, neural) [5]	Mostly English	Partly	No	Clusters	No
Topic detection and tracking [6]	Some	No	No	Yes	No
Crisis message clustering [7]	Some	No	No	Partly	No
CAD duplicate-call features	No	No	Human decides	No	No
This component (C4)	Yes (5 language styles)	Yes (evidence judge)	Yes (3:1 cost ratio)	Yes (incident level)	Yes

2.5 Gap Synthesis and Conceptual Framework

The review shows that the individual ideas needed by the component exist in the literature: field-wise comparison and clerical review from record linkage, near-duplicate trap cases from event coreference, on-line assignment against clusters from TDT, language-agnostic embeddings, NLI for contradiction, distillation for cheap extraction, and cost-sensitive decisions on calibrated probabilities. What is missing is their combination for multilingual, code-mixed emergency reports arriving over voice and SMS, together with an evaluation that separates the hard cases (same place at a different time, the same road with a different house number) from the easy ones.

The conceptual framework of the component follows from this synthesis. Each report is first turned into a reconciled record and a pool of verbatim evidence items. Four independent channels then score the report against each candidate incident: geography answers 'is it the same place?', time answers 'is it plausible that the

incident is still being reported?', semantic similarity answers 'is the overall story the same?', and the evidence judge answers 'do the individual facts agree or conflict?'. A fusion model learns how much to trust each channel, including when a channel is missing, and outputs a calibrated probability. A cost-derived threshold turns the probability into a decision, and uncertain or ambiguous cases are flagged for review. This structure keeps each channel interpretable and testable on its own, which is important for a safety-relevant decision.

3 RESEARCH METHODOLOGY

3.1 Research Design

The research follows a design science approach combined with controlled quantitative experiments. An artefact, the fusion component, was designed to address the problem in Section 1.3, built in increments, and evaluated after each increment against a fixed protocol. Each step of the component was treated as a small experiment with its own question, baseline, candidate methods and selection criterion, so that every design decision is supported by a measurement rather than by intuition.

Three principles governed the design of the experiments.

- Learned models before hand-written rules. Where a decision could be learned from data, a learned model was built and compared with the simplest rule-based alternative on the development split. A rule was kept only when the data could not support a model (for example, a phrase that never occurs in the corpus) or when the learned model was measurably worse.
- Development-only selection. Every choice of model, hyper-parameter, threshold and feature set was made on the development (DEV) split. For each adoption decision, the rule that would decide it was written down before the run, for example 'adopt if the DEV false merge rate does not rise and the false split rate falls'. The test (TEST) split was used once per adopted model to report the result, never to choose.
- Honest reporting. Results that went against expectations were reported as they were, including a DEV/TEST disagreement in the evidence judge and a design choice adopted by policy rather than by measurement. Where the TEST split had to be scored again after a later change, this is stated with a re-use note.

The evaluation also replays the live code path. Reports of a split are fed to the component in arrival order, each report is compared only with what had arrived before it, and the component builds its own incidents as it goes, so that an early mistake carries forward exactly as it would in operation. This replay is stricter than a pairwise evaluation, in which every pair is judged independently.

3.2 Data Participants or Study Materials

3.2.1 Source corpus

No human participants were recruited for this component. The study material is a purpose-built simulated corpus of emergency conversations that the author created manually for this component, consisting of voice call transcripts and SMS conversations in which an operator (or an automated dialogue agent) asks questions and a caller answers. The author wrote the conversations by hand to reflect the language mix of Sri Lankan callers, including Sinhala and Tamil in native script, English, and romanised code-mixed Singlish and Tanglish, and to include the situations that make fusion difficult: several reports of one incident from different channels and languages, delayed follow-up reports, and different incidents that share a place, a type or a day.

From this corpus the author built the C4 training-ready dataset, a workbook with one sheet per kind of record. Table 3.1 summarises its size. Splits were made by incident, never by report, so that no incident has reports in two splits; incidents that belong to a trap group were forced into the same split, because otherwise the confusable pair would be lost.

Table 3.1: Size of the C4 dataset (final version)

Item	Count	Breakdown
Incidents	175	Fire 85, Flood 83, insufficient information 4, non-emergency 3
Incidents per split	175	train 101, dev 34, test 40
Reports	535	call 270, SMS 265; train 308, dev 112, test 115
Reports by language	535	Sinhala 187, Tamil 145, Singlish 81, English 79, Tanglish 43
Conversation turns	3,501	operator and caller turns parsed from the transcripts
Evidence items (caller answers)	1,445	one per caller turn, with its operator question
Labelled report pairs	2,802	651 positive, 2,151 negative, 11 pair types
Trap groups	24	groups of confusable incidents (same place, road or day)
Evidence pairs for the judge	1,312	1,156 real + 156 synthetic house-number pairs

3.2.2 Report pairs and trap cases

The report pairs are the main evaluation material for the scoring channels. Positive pairs are every pair of reports inside one incident, subdivided into cross-lingual pairs (385), same-language pairs (92) and delayed pairs in which the second report arrives

much later (174). Negative pairs are of several kinds, ordered here from easy to hard: random pairs from different incidents (1,282), pairs reported on the same day with the same or a different type (548), pairs at the same place with a different time or type (214), and pairs of adjacent houses on the same road that needed separate dispatches (49). The last kind, called the adjacent-house trap in this thesis, is the hardest: the two reports share the road, the town, the type and the time, and differ only in the house number. Four such trap pairs were added to the test split specifically so that the claim that the component can separate two houses on one street could be measured on unseen data.

3.2.3 Reference labels and human agreement

The reference ('gold') fields for every report (incident type, location, number of people, trapped, injured and help needed) and the labels of the evidence pairs were drafted with the help of a large language model working from the caller's words alone, before any output of the component was seen. To avoid circular evaluation, the extractor used inside the component at run time was taken from a different model family. The drafted labels were then checked by a person in a blind agreement study: forty reports and forty evidence pairs, drawn with a fixed random seed and stratified by channel and language, were labelled independently before the drafted labels were revealed. Agreement was 89.4% on the gold fields and 77.5% on the evidence pair labels (Cohen's kappa 0.57), and 12 of 12 decisive conflicts (different house or place) were labelled the same. The check covered Sinhala, Singlish and English only, because those were the languages the checker reads; Tamil and Tanglish labels were not independently verified.

The check also found systematic errors, which were corrected in new dataset versions: in 208 reports the number of people had been copied from the number trapped, and 699 evidence pairs had been labelled as corroborating only because the incident type matched, although many came from different incidents. A further 49 evidence labels and two count decisions were set by the researcher. Table 3.2 lists the dataset versions; each version was built by a script from the previous one and the previous file was never edited.

Table 3.2: Dataset versions and the reason for each change

Version	Change	Reason
v2	Dataset as first assembled	Starting point
v3	People count reset to unknown in 208 reports	Count had been copied from the trapped count (found by the human check)
v4	Evidence pairs relabelled from separate type and place facts; 49 researcher labels	Type match had been taken as agreement
v5	Evidence pairs aligned to the runtime evidence; 408 duplicates merged; count differences relabelled count_variance	Training text must equal runtime text; counts are not contradictions
v6	156 synthetic same-road, different-house-number evidence pairs added	The judge read house numbers as counts
v7	Spelling-variant and identity pairs added	Tested and not adopted (Section 6.1)

3.2.4 External resources

Three external resources were used to build labels or training data, never as an input that decides at run time without measurement. Google Places was used to resolve the gold locations to coordinates, from which the place model's labels were derived (same place within 0.5 km, near between 0.5 and 5 km, far beyond 10 km). The GeoNames gazetteer for Sri Lanka (42,595 sites, licensed CC-BY 4.0) provided extra place-name pairs for pre-training the place model. A large language model (GPT-4o mini, version gpt-4o-mini-2024-07-18, pinned) was used as the teacher whose guarded outputs on the training split were distilled into the mT5-small extractor.

3.3 Experimental or Study Procedure

The procedure for each step of the component followed the same sequence.

- Define the step's question and its selection criterion on DEV, and write the adoption rule.
- Build the simplest baseline (for example, character n-gram TF-IDF with logistic regression, an off-the-shelf encoder, or the existing rule).
- Train candidate models on TRAIN, select settings on DEV, and, where the result must be compared with the baseline, apply the adoption rule.
- For models trained on a GPU (on Google Colab), produce a health report and reproduce the result on the development laptop before use.
- Report TEST once for the adopted model and save every table and chart produced by the run.

The health report for GPU-trained models consisted of the following checks, all of which had to pass: the run used a GPU in full precision with no numerical errors; the training loss decreased; the model could over-fit a tiny subset (a sanity check that the pipeline can learn); a control with shuffled labels did not beat the baseline (a check against leakage); the model beat the baseline on DEV; three random seeds agreed within a fixed tolerance; and no example appeared in more than one split. The laptop verification then checked the SHA-256 hash of the model files, confirmed that the data were identical, and recomputed the DEV and TEST outputs on the CPU, which had to match the Colab outputs to within a small tolerance (for example, a maximum difference of 0.000011 in the judge's probabilities).

For the decision steps (fusion, decision and incident building), the procedure added a cross-fitted replay. The DEV incidents were divided into five folds and each report was scored by a fusion model that had never seen its incident. Because a single division into folds turned out to be optimistic, later decisions were re-checked as the mean over ten different random fold divisions, with the adoption rule applied to the mean (Section 6.1).

3.4 Measures and Evaluation Metrics

Different steps were measured with metrics suited to their outputs. Table 3.3 lists them.

Table 3.3: Evaluation metrics used for each part of the component

Part	Metric	Meaning
Extraction (Step 1)	Field accuracy vs gold	Share of reports whose reconciled value equals the gold value
Classifiers (readers, Model C)	Accuracy, macro-F1	Macro-F1 weights every class equally
Scoring channels (3B, 4, 5A, 5B)	ROC AUC on report pairs	Probability that a positive pair scores above a negative pair
Evidence judge (5B)	Macro-F1 over 4 labels	corroborates / contradicts / neutral / count_variance
Fusion (Step 6)	Log loss, Brier score, AUC, ECE	Quality and calibration of P(same incident)
Candidate cap (Step 4)	Recall@K	Share of true links kept among the top K candidates
Decision and incidents (7-9)	False merge %, false split %, link recall %, merge precision %, review %	Operational error rates in the replay
Incidents (7-9)	B-cubed precision, recall, F1	Clustering quality of the incidents built

Part	Metric	Meaning
Hard cases	Adjacent-trap merges	Number of adjacent-house trap pairs wrongly merged

The operational rates are defined as follows. The false merge rate is the share of all reports that were merged into an incident they do not belong to. The false split rate is the share of linkable reports (reports whose incident was already open when they arrived) that were not merged into it. Link recall is the share of linkable reports that were merged correctly, merge precision is the share of merges that were correct, and the review rate is the share of reports flagged for human review. B-cubed precision and recall [31] average, over reports, the purity and completeness of the incident that each report ends up in, and are a standard measure for evaluating clusters of coreferent items.

3.5 Data Analysis

Model training and analysis used Python with scikit-learn [32] for logistic regression, histogram gradient boosting [33] and calibration, and PyTorch with the Hugging Face Transformers library [34] for the neural models. Selection over hyper-parameters used DEV performance or, where DEV was too small for a single estimate, five-fold cross-validation repeated five times on DEV. Differences that were smaller than the spread across folds or seeds were treated as ties, and in a tie the simpler or more conservative option was kept.

Because the DEV and TEST splits are small (112 and 115 reports, of which 78 and 75 are linkable), a single report changes a rate by about one percentage point. Results are therefore given with their counts, and the main DEV results are given as the mean and standard deviation over ten fold divisions. Breakdowns by language, channel and pair type were used to find where each model fails, and every adoption decision was recorded in the project documentation together with the numbers that supported it.

3.6 Ethics Privacy Security and Responsible AI

The study used simulated conversations only, so no personal data of real callers were processed by this component. The component was nevertheless designed as if it

would handle real emergency reports. Reports are stored locally and are not sent to third parties except where a feature explicitly requires it: a location string may be sent to the Google Places API for geocoding and, when the corresponding option is enabled, a place string may be sent to the language model for reading a house number. Both calls are switchable and both are cached so that the same string is sent only once. API keys are read only from an environment file that is excluded from version control, and never appear in code, logs or data files.

Several design decisions follow from responsible AI considerations. The caller's words are kept verbatim as evidence and are never rewritten by a model, so that what a dispatcher sees is what the caller said. Generative models are confined to places where their output is checked: the runtime extractor's location must be a span of the caller's own text, and any count it outputs must agree with the number in its supporting quote. The merge decision does not use urgency or priority, which prevents a plausible but irrelevant signal from influencing who receives a response. Uncertain and ambiguous cases are flagged for a person rather than resolved automatically, and every merge carries an evidence trail that explains it.

3.7 Reproducibility and Research Artefacts

All code, configuration and selection scripts of the component are kept in the group's version-controlled repository. Each experiment is a script that reads a named dataset version and writes its tables, metrics and charts to an output folder, so that every number in this thesis can be traced to the script and data that produced it. Every installed model has a provenance file with its training run, data hash and selection result. Random seeds are fixed, the language model version is pinned (gpt-4o-mini-2024-07-18), and responses from external services are cached, so that re-running an evaluation does not depend on the current behaviour of a remote service. The automated test suite (Section 5.4) checks the invariants of every step.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements were derived from the problem statement, from the needs of a dispatcher and from the interface agreed with the other components. Table 4.1 lists the functional requirements and Table 4.2 the non-functional requirements.

Table 4.1: Functional requirements of the fusion component

ID	Requirement
FR1	Accept finished reports from C2 (voice) and C3 (SMS) through a versioned contract, and reject a report that does not match the contract.
FR2	Reconcile the upstream fields with the component's own reading of the conversation, keeping every source value.
FR3	Keep the caller's words verbatim as evidence; never translate or rewrite them.
FR4	Score each report against every open incident on geography, time, semantic similarity and evidence agreement.
FR5	Produce a calibrated probability that a report and an incident are the same.
FR6	Decide MERGE or SEPARATE with a cost-derived threshold, and flag uncertain or ambiguous cases for review.
FR7	Maintain a fused incident record with consolidated fields, a timeline and an evidence trail for every link.
FR8	Expose reports and incidents through an API and show them on an operator page and a dashboard.
FR9	Carry urgency and priority through for display, without using them in the merge decision.

Table 4.2: Non-functional requirements of the fusion component

ID	Requirement	Target
NFR1	Response time to C2 and C3	Reply within the callers' 20 s budget; heavy steps after the reply
NFR2	Safety of the decision	False merges close to 1% of reports; adjacent-house traps not merged
NFR3	Robustness to missing input	A missing channel is recorded as missing, never as a zero score
NFR4	Graceful degradation	If an external service or model is unavailable, the remaining sources carry on
NFR5	Explainability	Every merge has an evidence trail with scores and the aligned caller words
NFR6	Security	API keys only in an environment file; no keys in code, data or version control
NFR7	Cost	Runs on a laptop CPU at inference; external calls cached and switchable
NFR8	Reproducibility	Pinned model versions, fixed seeds, scripted experiments, provenance files

4.2 Component Architecture within the Complete Solution

Within the group's solution, C4 is the last component before the dispatcher. C2 and C3 publish each finished conversation to C4 over HTTP using a shared client, and C2 attaches the urgency verdict of C1 to each call. C4 stores the report, processes it and keeps the set of incidents up to date. The dispatcher sees the result on the incident dashboard.

Inside C4 the processing is organised in nine steps (Figure 4.1). Step 1 turns the incoming report into a reconciled record. Step 2 builds the evidence pool from the caller's answers. Steps 3, 4, 5A and 5B are four independent scoring channels that compare the report with each candidate incident. Step 6 fuses their outputs into a calibrated probability, Step 7 decides, Step 8 updates the incident and its record, and Step 9 flags ambiguous cases. The channels are trained separately and combined late, so that each can be tested, replaced or switched off without retraining the others.

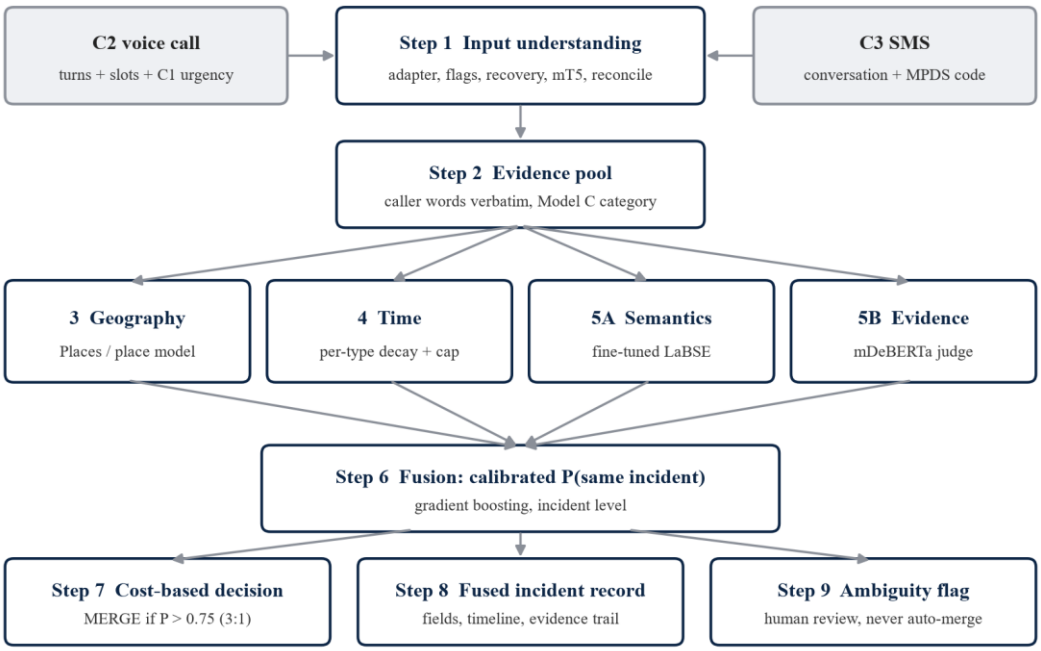


Figure 4.1: The nine processing steps of the fusion component (C4)

4.3 Design Alternatives and Decisions

Several important design decisions were taken by comparing alternatives on DEV. Table 4.3 summarises the main ones; the measurements behind them are given in Chapter 6.

Table 4.3: Main design alternatives and the option chosen

Decision	Alternatives considered	Chosen	Reason
Runtime extractor	GPT with guard; mT5-small student; XLM-R span extractor	mT5-small student	No API, runs on CPU; DEV 0.788 vs upstream 0.155; XLM-R 0.50
Evidence text	English propositions; verbatim caller words	Verbatim	Translation can remove the decisive detail
Place comparison	Hand-written phonetic rules; learned place model	Learned, rule-free	DEV macro-F1 0.777 vs 0.720
Geo scorer	Logistic regression; gradient boosting	Logistic regression	Better calibrated (DEV Brier 0.094 vs 0.096)
Time model	Fixed window; learned decay per type	Spline over log gap, per type	Floods last longer than fires; learned from data
Semantic encoder	TF-IDF; off-the-shelf LaBSE; fine-tuned LaBSE	Fine-tuned LaBSE	DEV AUC 0.970 vs 0.833
Evidence judge	TF-IDF + LR; LaBSE + LR; fine-tuned mDeBERTa	Fine-tuned mDeBERTa (XNLI base)	Best DEV macro-F1, seeds stable
Fusion	LR trained on TRAIN + Platt; LR on DEV; boosting on DEV	Boosting fitted on DEV	Channel scores are in-sample on TRAIN
Matching unit	Earlier report; whole incident	Whole incident	False split 24.0% to 10.7% on TEST
Revision	Retrospective merge; detach; flag only	Flag only	Retro merges were wrong 3 of 4 times

Two decisions deserve a short explanation here. First, the component deliberately avoids translating the evidence. An early design wrote each caller answer as an English proposition, but a translation can normalise away a house number, a negation or a place name, and it adds a model whose errors are hard to see. The final design keeps the answer exactly as spoken and pairs it with the operator question it answers. Second, the component prefers learned models to hand-written rules. Hand-written rules were kept only where the corpus contains no example to learn from, for instance words such as 'our side' or 'here' used in place of a location, and Sinhala and Tamil number words that never occur in the data.

4.4 Detailed Design

4.4.1 Step 1: input understanding

The input adapter turns the two report schemas (call and SMS) into one internal record. It parses the conversation into ordered turns, separates operator and caller turns, converts timestamps, and maps the upstream fields (incident type, location, counts, priority) into a common form. Values are stored as sent. A missing value is recorded as missing and never as zero, because 'nobody is trapped' and 'the caller did not say' must lead to different decisions. The language of the report is identified by a

small learned classifier over character n-grams of the caller's words (DEV accuracy 0.973, TEST 1.000); language is used only for display and analysis.

Step 1b computes quality flags for the upstream values, for example whether the location is present, how specific it is (address, road, landmark, area) and whether it is suspect, such as a whole sentence or a count mistaken for a place. Step 1c recovers values from the conversation itself: it classifies each operator question (location, people, trapped, injured, confirmation and so on), takes the caller's next turn as the answer and reads it with a multilingual number reader that understands Sinhala, Tamil and romanised number words, compound forms and expressions such as 'me plus ten others'.

Step 1d extracts the fields with a learned model. A large language model (GPT-4o mini, pinned version) was used as a teacher on the training split only, inside a six-check guard that accepted a value only if it was supported by a verbatim quote from the caller. The accepted outputs were distilled into a google/mt5-small model fine-tuned on a Colab T4 GPU. At run time the student needs no API and runs on the laptop CPU in about three seconds per report. Two checks remain: the location must be a span of the caller's own words, and any count must agree with the number in its quote. Learned readers further check the incident type and the help requested against the caller turns that actually state them.

Step 1e reconciles the three sources (the student extractor, the recovered values and the upstream values) field by field. When all three agree the value is taken with high confidence; when two agree the majority is taken with medium confidence and the dissenting value is recorded; a single source gives low confidence; and when no two sources agree the value is shown by priority but marked as having no confidence, in which case the location is not used for geography. Two place strings in different scripts are compared with the learned place model of Step 3.

4.4.2 Step 2: evidence pool

Step 2 builds the evidence pool of the report: one item per caller turn, consisting of the caller's words verbatim and the operator question they answer. Each item is given a category (place detail, people count, life safety, resource need or incident nature)

by Model C, a logistic regression over frozen LaBSE embeddings and character n-grams of the question and the answer. Model C reached a DEV macro-F1 of 0.982 and a TEST macro-F1 of 0.996. No word list, template or translation is used in this step.

4.4.3 Step 3: geography

Step 3A resolves the location to coordinates where this is reliable. A Latin-script location is sent to the Google Places API (with a cache) and, if a result inside Sri Lanka is returned, the report gets coordinates. A measurement on the corpus (Section 6.1) showed that Places is much less reliable for Sinhala and Tamil script, and that a wrong coordinate is worse than none, so in the evaluated configuration native-script locations are not sent and the report keeps its location as text (text fallback). A suspect or disputed location leads to abstention, and a report without a location is marked as such.

Step 3B scores the geographic compatibility of a report and a candidate. When both have coordinates the score uses the distance and the size of the area that Places returned. Otherwise it uses a learned place model, which takes two place strings in any script and outputs the probabilities that they are the same place, nearby places or far apart. The place model uses frozen LaBSE embeddings and character n-gram TF-IDF of the two strings, with interaction features that let it learn which shared or differing parts matter, such as a house number or a road name. Sinhala and Tamil strings are read through standard IAST transliteration so that cross-script pairs share characters. The model was pre-trained with weight 0.3 on place-name pairs from the GeoNames gazetteer. A logistic scorer, calibrated with Platt scaling, combines these inputs into one score. A missing location gives no score and a missing-channel flag, never a score of zero.

4.4.4 Step 4: time and the candidate cap

Step 4 estimates whether the time gap between a report and the latest report of a candidate incident is plausible for the same incident. Instead of a fixed time window, the probability is learned with a logistic regression over a spline of the logarithm of the gap, fitted separately for floods, for fires and for all types together. Figure 4.2 shows the learned curves: the probability that a report eight hours later belongs to the

same flood is 0.086, against 0.042 for a fire, because floods last longer. The type of the report and of the candidate never removes a candidate; it is passed to fusion as features.

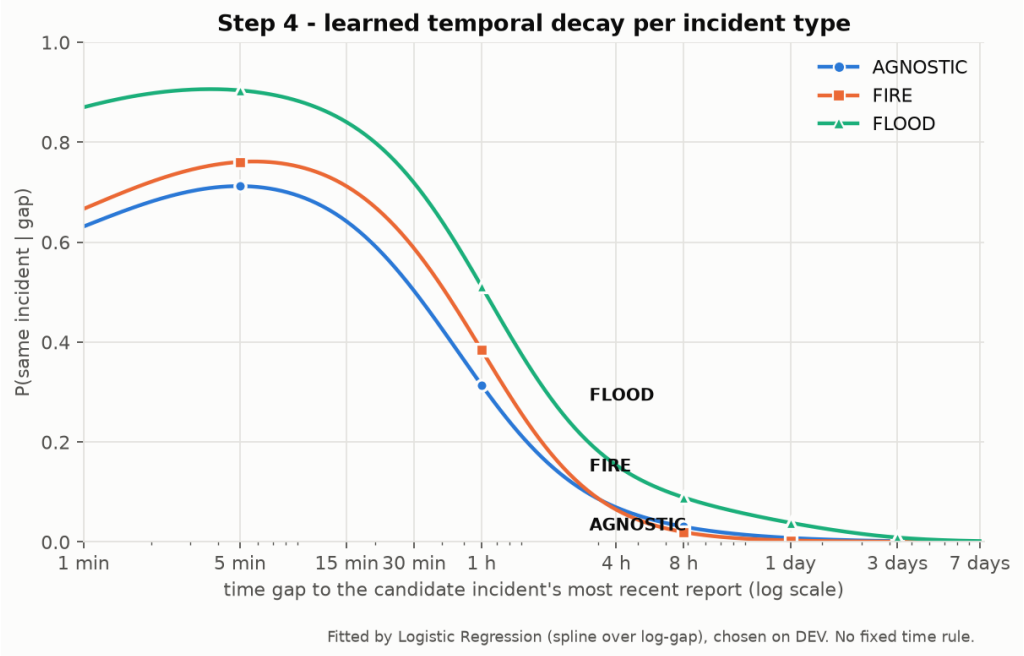


Figure 4.2: Learned probability that a report belongs to the same incident as a function of the time gap, per incident type

The geography and time scores are then combined by a small learned model that ranks the candidates, and only the top K are passed to the expensive steps 5A and 5B. Every candidate remains in the output, marked as kept or not kept. The value of K was selected on a stress replay in which every incident of a split is open at once, the worst case the corpus allows: K is the smallest value that keeps every true DEV link. The selected value was K = 23, with no reserved places for type mismatches.

4.4.5 Step 5: semantic similarity and evidence matching

Step 5A measures the overall similarity of what the two callers said. The caller turns of each report are embedded with LaBSE fine-tuned on the C4 report pairs using a contrastive (CoSENT) objective with hard negatives, and the cosine similarity is the score. Operator turns are excluded, because the operator asks the same questions in every call.

Step 5B compares individual facts. It has three stages. Align pairs each evidence item of the incoming report with the items of the candidate whose answers are similar

(fine-tuned LaBSE cosine of at least 0.5, chosen on DEV). Judge classifies every aligned pair with a fine-tuned mDeBERTa-v3 model as corroborates, contradicts, neutral or count_variance; each side is the caller's answer followed by the operator's question, and the prediction is the average over both orders of the pair. Summarise turns the judgements into features: the number of corroborations, contradictions and count differences, the number of new items, the number of aligned pairs and the maximum probabilities of corroboration and contradiction. The aligned pairs are kept verbatim for the evidence trail.

4.4.6 Steps 6 and 7: fusion and decision

Step 6 builds one feature vector per report and candidate: the geography score and its missing flag, whether both sides have coordinates, the place model's same and far probabilities, the time score and the log gap, type agreement features, the 5A cosine and its missing flag, the 5B summary features, and two house-number features (same number, different number) read from the reconciled locations. Missing values stay missing, with a flag. A histogram gradient boosting classifier of depth three turns the vector into P(same incident). It is fitted on the DEV split rather than on TRAIN, because every channel model was trained on TRAIN and its scores there are in-sample and over-confident (for example, the 5A AUC is 1.000 on TRAIN and 0.970 on DEV).

The fusion model's training data were augmented in two ways to teach it cases the real data do not contain. Synthetic adjacent-house reports are real reports with only the house number changed, processed through every live step and labelled as different incidents. Counterfactual rows are copies of real same-incident examples with the type features, or the house-number features, flipped to 'different' and labelled as different. This teaches the model that a confirmed fire and a confirmed flood, or two known different house numbers, are not the same incident, without a hand-written rule.

Step 7 merges the report into the best candidate when its probability exceeds the threshold derived from the costs of the two errors:

MERGE if P(same) > C_FM / (C_FM + C_FS); with C_FM : C_FS = 3 : 1 the threshold is 0.75

Otherwise the report opens a new incident. A report that was separated although its best probability was between 0.5 and the threshold is flagged for review, because the cost made the component separate a link that was more likely than not. Two policy rules set by the researcher are applied before the decision: a report without a location is never merged, and nothing is merged into an incident that has no location. A rule that fires and floods are never merged was built and measured but is switched off, because the fusion model learned the same behaviour from the counterfactual rows.

4.4.7 Steps 8 and 9: incident record and revision

Step 8 makes the incident, not the single earlier report, the unit of comparison. The geography score of an incident is that of its best-matching member, the time gap is measured to its latest member, its type is the majority type of its members, and the evidence judge compares the incoming report with the pooled evidence of all members. When a report is merged, the fused incident record is updated: fields on which the members agree, counts that differ shown as a range, disagreeing values kept with their sources, a severity equal to the highest member priority (shown only), a timeline of what each report added, and the evidence trail of every link.

Step 9 handles cases where a report scores above the threshold for two or more incidents. Retrospective merging of such incidents and detaching recent members were implemented and measured but not adopted (Section 6.1). Instead, the report is flagged for review with a note that the incidents may be one, and the decision itself is not changed. The dashboard shows each flagged incident next to the incident it may match.

4.5 Standards and Professional Practice

The component follows several standards and practices. The data contract with C2 and C3 is a versioned schema (c4.v2) validated with Pydantic; unknown fields are rejected so that a change on one side cannot pass silently. The service exposes a REST interface with JSON over HTTP. Severity follows the MPDS determinant codes sent by C3 and the urgency levels sent by C1, and both are displayed exactly

as received. Native-script text is handled as Unicode throughout and transliteration follows the IAST scheme. Geographic data from GeoNames are used under the CC-BY 4.0 licence with attribution, and map tiles on the dashboard are from OpenStreetMap with attribution.

Professional practice was followed in the development process: version control for all code, automated tests that run before any change is accepted, separation of configuration and secrets from code, scripted and reproducible experiments, and written records of every design decision together with the evidence for it. The evaluation followed the common practice in machine learning of selecting on a validation split and reporting a test split once, and the instances where this practice was strained are disclosed.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The component is written in Python and consists of about 23,000 lines of code organised in one package per step (adapter, extraction, evidence, geo, temporal, semantic, matching, fusion, decision, incidents, revision) and a service package for the live system. Table 5.1 lists the main technologies.

Table 5.1: Technology stack of the fusion component

Layer	Technology	Use
Language and runtime	Python 3.11	All component code
Data models	Pydantic 2	Internal report record, API contract c4.v2
Classical ML	scikit-learn	Logistic regression, histogram gradient boosting, calibration, TF-IDF
Deep learning	PyTorch, Hugging Face Transformers, sentence-transformers	mT5-small, LaBSE, mDeBERTa-v3
Training hardware	Google Colab (NVIDIA T4 GPU)	Fine-tuning; laptop CPU for all inference
Transliteration	indic-transliteration	Sinhala and Tamil script to IAST
Geocoding	Google Places API (cached)	Coordinates for Latin-script locations
Teacher model	OpenAI GPT-4o mini (gpt-4o-mini-2024-07-18)	Distillation targets on the training split
Web service	Flask	REST API, operator page, dashboard
Front end	HTML, JavaScript, Leaflet, OpenStreetMap	Operator page and incident dashboard
Data files	openpyxl, JSON Lines	Dataset workbook, report store, outputs
Testing	pytest	421 automated tests

Development took place on a Windows laptop without a GPU. All inference, including the three transformer models, runs on its CPU. Training of the transformer models was done on Google Colab, and every trained model was copied back and verified on the laptop before it was installed (Section 3.3).

5.2 Component Implementation

5.2.1 Processing pipeline

Each step is implemented as a pure function from a report record to an enriched report record, which keeps the steps independent and easy to test. The steps never overwrite an earlier value: Step 1b adds flags, Step 1c adds recovered values, Step 1d

adds extracted values, and Step 1e adds the reconciled fields while keeping every source value. The dataset labels are stored in a separate metadata field that no processing step is allowed to read; a test checks this. The trained models are loaded once and cached, and the results of expensive operations (student extraction, judge probabilities, Places lookups) are cached by input text so that repeated text costs nothing.

5.2.2 Live service

The live service receives a report, runs Steps 1 to 3A and replies at once with the reconciled record and the decision marked as pending. A background worker then runs Steps 3B to 9 against the open incidents in arrival order, attaches the report to an incident and updates the incident store. This split was introduced after measuring the timing: with the evidence judge inside the reply, a report with seven kept candidates and nineteen aligned pairs took about 12.7 seconds in total, too close to the 20-second budget of C2 and C3 and growing with the number of open incidents. After the change the reply takes about four seconds (of which the student extractor takes about 3.4 s), and the evidence judge finishes 0.9 to 2.3 seconds later in a single batched call. The incident store is written to disk after every decision and survives a restart.

5.2.3 Operator page and incident dashboard

Two web pages were built on top of the API. The operator page (Figure 5.1) is a testing view that lists the live reports and, for a selected report, shows the result of every step: the input adapter, the final Step 1 decisions with their confidence, the evidence pool with Model C categories, the location resolution, the scores of every candidate, the decision and the incident. It was used throughout development to inspect live cases.

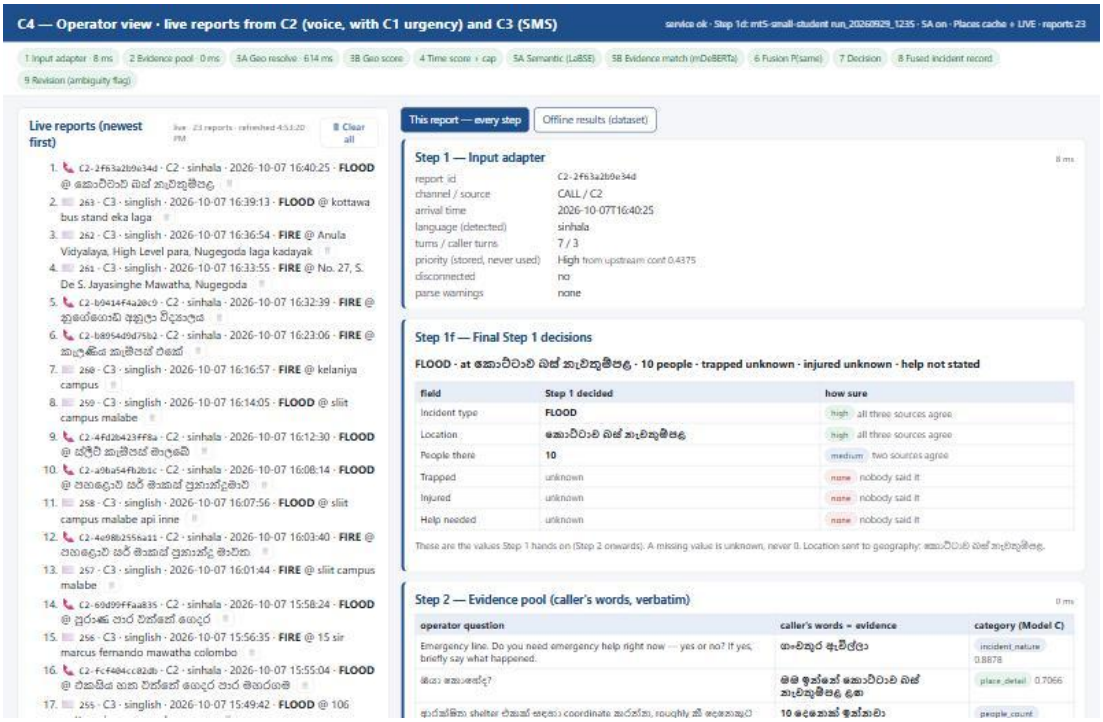


Figure 5.1: Operator testing page: live reports from C2 and C3 (left) and the output of each processing step for the selected report (right)

The incident dashboard (Figure 5.2) is the view intended for a dispatcher. It shows key figures (incidents, reports fused, incidents reached by both calls and SMS, merges, reviews required, incidents without a location), a map of incidents with marker size proportional to the number of reports and colour by type, Venn diagrams of the channels that reached each incident and of the channels that agreed on each merge, a timeline, the decision confidence of every report, and a searchable table of incidents. An incident flagged for review shows the incident it may match next to it. Selecting an incident opens its fused record with the consolidated fields, the timeline, the evidence trail with the aligned caller words, and the scores of each merge. The dashboard reads the API every five seconds and works in light and dark themes and at phone width.

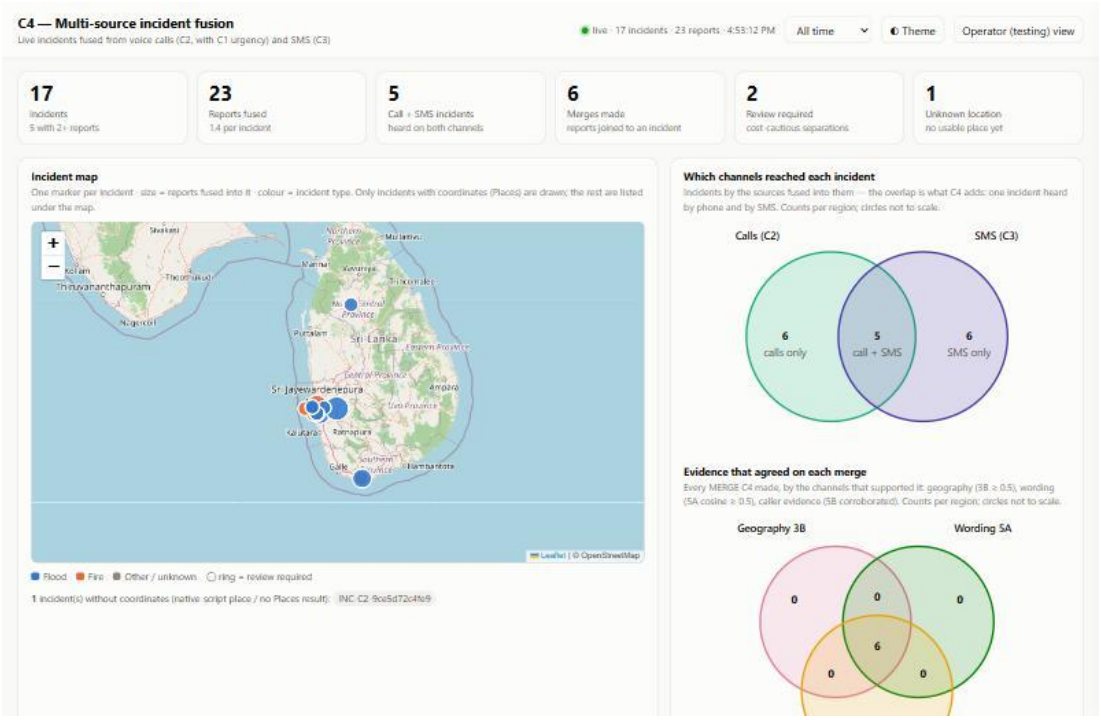


Figure 5.2: Incident dashboard with key figures, incident map and channel overlap

5.3 Integration with the Other Components

Integration with C1, C2 and C3 uses a versioned contract, c4.v2, implemented as a Pydantic model. A report carries the ordered turns of the conversation (operator and caller), the upstream fields in their native form, the C1 urgency verdict for calls or the MPDS code for SMS, a flag for a dropped call, and identifiers for a case that supersedes an earlier one. The contract forbids unknown fields, so a change on either side is reported as an HTTP 422 error rather than being silently ignored. Figure 5.3 shows the flow.

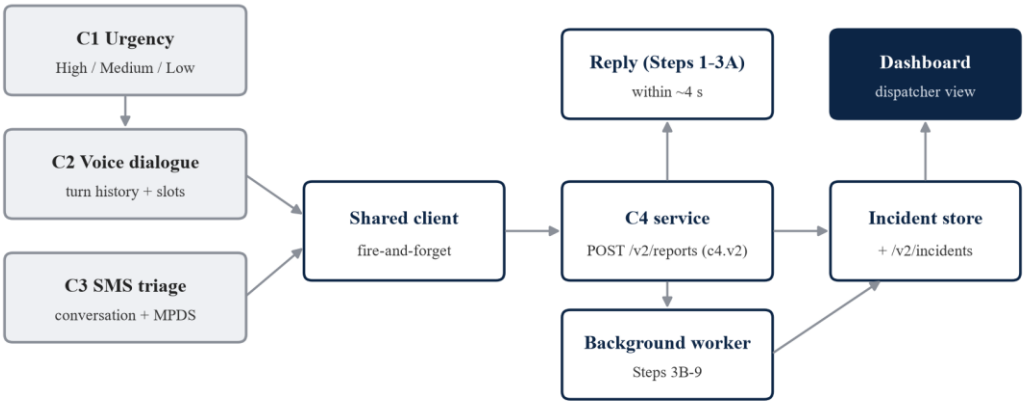


Figure 5.3: Integration of the fusion component with C1, C2 and C3

A small number of changes were needed on the other components' side and were agreed with their owners: the shared client was rewritten for the new contract and remains fire-and-forget, so that a slow or unavailable C4 never delays a caller; C2 sends its full turn history, the opening question and the complete C1 urgency verdict; and C3 sends its full conversation log rather than the last five turns, and also publishes a partially completed case when an SMS conversation is abandoned. An integration check script starts a real C4 server and publishes C2 and C3 sessions through the real shared client, verifying the complete path end to end.

5.4 Verification and Software Testing

Testing was carried out at four levels.

- Unit tests. The automated suite contains 421 test cases in 25 test modules (194 test functions, many of them parameterised), covering every step. Typical tests check that a missing value stays missing, that an unanswered question never borrows a later answer, that the evidence text equals the caller's words character for character, that a count difference is never labelled a contradiction, that a missing channel never becomes a zero score, and that the fusion feature vector contains no urgency or priority field. The full suite passes (421 passed, 1 skipped).
- Guard tests. A corruption test deliberately damaged correct extraction outputs (wrong numbers, invented places, quotes that do not occur in the text) and measured how many the guard rejected, before any paid API call was made.

- Model verification. Every GPU-trained model passed the health report and the laptop reproduction described in Section 3.3 before installation. Two training runs were rejected by these checks: a first evidence judge run whose three seeds disagreed (macro-F1 0.709, 0.820, 0.741), and a later judge run that passed its health report but made the fusion model worse end to end, so it was not adopted.
- Integration and live tests. The integration script exercised the C2 and C3 clients against a real server. In addition, scripted live scenarios were entered through C2 and C3 to test hard cases, such as the same address in two scripts, two houses on the same road, a report without a location and a menu answer recorded as the type.

The live scenarios were also a source of new test cases. When a live case exposed a fault, such as a Sinhala teen number ('එකොළොස්', eleven) that was not read, a house number read by the judge as a count, or 'අපේ පැත්තට' ('to our side') being geocoded as a place, the fault was reproduced in a test, a fix was designed, and the fix was measured on DEV with a rule written beforehand, so that a correction for one live case could not silently damage the measured results. Table 5.2 lists examples.

Table 5.2: Examples of faults found in live testing and how they were handled

Live case	Fault	Fix	DEV check
'එකොළොස් දෙනෙක් ඉන්නවා' (eleven people)	Teen number not read	Teen numbers 11-19 added to the number reader	No count changed on DEV or TEST
'23 පන්සල පාර' vs '14 පන්සල පාර'	House number read as a count	Synthetic house-number pairs (v6) and house features in fusion	Adopted by DEV rules
'අපේ පැත්තට' as location	Deictic phrase geocoded	Treated as no location	0 corpus strings changed
Menu answer 'ගිනි' with type FLOOD	Operator words taken as the caller's	Learned type reader checks the type quote	DEV type 107 to 109 of 112
FLOOD SMS merged into a FIRE incident	Type disagreement not learned	Counterfactual type rows in fusion	0 FIRE/FLOOD merges on DEV

5.5 Deployment and Operation

The component runs as a single Flask service on port 8000. It needs Python, the packages listed in its requirements file and the installed model files (about 4 GB, almost all of it the three transformer models); no GPU is needed. Secrets are read from an environment file in the repository root, which is excluded from version control. Behaviour is controlled by environment switches, for example whether

uncached locations may be sent to Places at run time, whether native-script locations are sent to Places, whether the house-number reader may call the language model, and whether the policy rules are on. The offline evaluation scripts and the tests always run with external calls switched off, so the reported results do not depend on a remote service.

In operation, the dispatcher uses the dashboard, while the operator page serves for inspection and testing. A report that C4 cannot process because of a missing model or an external failure is still stored and shown; the affected channel is recorded as missing and the decision is made by the remaining channels. Because every decision, its scores and its evidence trail are stored, a supervisor can audit any merge after the event.

6 RESULTS AND VALIDATION

This chapter reports the results in the order of the processing steps. Unless stated otherwise, DEV numbers were used to make choices and TEST numbers are reported once for the adopted configuration.

6.1 Research Results

6.1.1 Step 1: extraction and reconciliation

The distilled mT5-small student was compared with the upstream extraction on the mean share of correct values over five fields (type, people, trapped, injured and help). The adoption rule, written before the comparison, was to use the student if it beat the upstream extraction on DEV with no field worse. The student scored 0.788 against 0.155 on DEV with no field worse and was adopted; on TEST it scored 0.73 against 0.12. A span-extraction alternative based on XLM-R reached only 0.50 on DEV and was not used. The guarded teacher model remained better than the student on every field except the incident type, which shows the cost of removing the API from the run time path. The check that the student's location must be the caller's own words removed 10 of 111 DEV and 17 of 112 TEST locations that the student had written in other words.

Table 6.1 shows the accuracy of the final reconciled fields (Step 1e consensus compared with the gold fields) in the live configuration. Several smaller learned components contributed to these numbers. A learned check of whether the caller asked for help raised the exact accuracy of the help field on DEV from 0.536 to 0.955, because the student often guessed the help from the incident type. A three-class type reader (fire, flood, other) that checks the student's type against the caller's own words raised the DEV type accuracy from 107 to 109 of 112 reports; on TEST the three-class reader gave 104 of 115 against 107 for the earlier two-class version, a cost that is reported as it was found.

Table 6.1: Accuracy of the reconciled Step 1 fields against the gold fields (% of scorable reports)

Split	Incident type	People	Trapped	Injured	Help needed
DEV	97.3 (109/112)	95.0 (19/20)	100 (72/72)	100 (48/48)	96.6 (28/29)

Split	Incident type	People	Trapped	Injured	Help needed
TEST	91.2 (104/114)	96.3 (26/27)	98.1 (51/52)	100 (53/53)	91.2 (31/34)

6.1.2 Step 2: evidence pool

Step 2 was checked first for its central requirement, that the evidence is the caller's own words. On the full corpus run, all 1,635 caller turns became 1,635 evidence items, no answer text was changed and no operator question was changed; after duplicate call openings were removed in dataset v5, 1,445 evidence rows remained, each identical to the evidence the live code builds. Automated tests confirm that Step 2 uses no word list, template, regular expression or translation; the only structure it keeps is the pairing of a caller turn with the operator question immediately before it.

Model C, which assigns each evidence item one of five categories (place detail, people count, life safety, resource need, incident nature), was selected on DEV: question and answer together as input, regularisation C = 0.3. Retrained on dataset v5, it reached a macro-F1 of 0.982 on DEV (0.973 for the earlier v4 model on the same rows) and 0.996 on TEST, and 97.7% and 99.0% category accuracy end to end through the pipeline on DEV and TEST. On the v4 rows the learned model had also matched or beaten the original rule-based category writer on TEST (macro-F1 1.000 against 0.992), so the rules were kept only as a baseline. These figures are high partly because the operator questions in the corpus come from a fixed set of templates; on romanised Singlish questions that do not occur in the data, Model C was less certain (probabilities between 0.32 and 0.58) and in one live test exchanged the people count and life safety categories. Only the place detail category decides anything downstream, in the evidence alignment of Step 5B.

Two further learned readers, Model A (operator question to slot) and Model B (caller answer to value type), were trained to replace the question and answer rules of Step 1c. Table 6.2 compares them with the rules on TEST. Model A was slightly better than the rules, but Model B was weaker on accuracy and, end to end, turned four answers such as 'people are trapped' into a count of zero, a false zero that the rules never produced. The rules therefore stayed in charge, and the two models were used as a second opinion: where they disagree with the rules, a disagreement flag is passed

to fusion and no value is changed. On TEST the rules were wrong in 3 of 21 flagged fields (14.3%) but in only 2 of 121 unflagged fields (1.7%), so the flag marks about eight times the error rate and gives the fusion model a useful signal of how far a field can be trusted.

Table 6.2: Step 2 and Step 1c readers on TEST: learned models against rules

Reader	Task	Learned model	Rules
Model C (evidence category, 5 classes)	Question + answer to category	Macro-F1 0.996 (v5); 1.000 on v4 rows	Macro-F1 0.992 (v4 rows)
Model A (6 classes)	Operator question to slot	97.9% / macro-F1 0.984	97.5% / macro-F1 0.979
Model B (4 classes)	Caller answer to value type	86.8% / macro-F1 0.773	88.2% / macro-F1 0.646
Step 2 end to end	Category of every evidence item	DEV 97.7%, TEST 99.0%	-

6.1.3 Step 3: geography

The Places measurement compared the coordinates returned for the gold location of each report with the true place. Figure 6.1 shows the result by script. For Latin-script locations, 83% were resolved to within 1 km; for Sinhala and Tamil script only 53% and 58%, and 12 to 13% were placed more than 25 km away, often confidently. Because a confident wrong coordinate can merge two distant incidents, native-script locations are compared as text by the place model in the evaluated configuration.

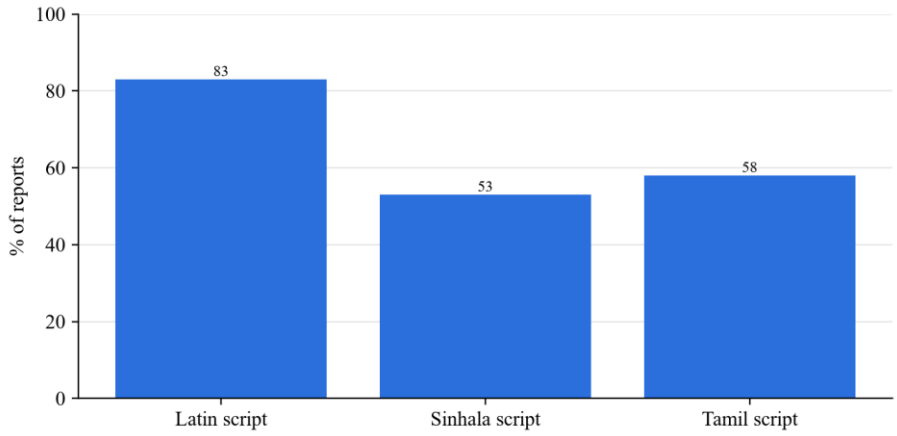


Figure 6.1: Share of gold locations that Google Places resolved to within 1 km of the true place, by script

The learned, rule-free place model was compared on DEV with an earlier version based on hand-written phonetic rules (a phonetic skeleton, number words converted

to digits and road-name patterns). The learned version reached a macro-F1 of 0.777 against 0.720 and replaced the rules. Pre-training on GeoNames place pairs with weight 0.3 was chosen on DEV (3B AUC 0.932 against 0.926 without). Figure 6.2 shows the TEST AUC of the Step 3B score by kind of pair. With coordinates on both sides the score reaches 0.973; in text fallback it reaches 0.838, and pairs in which both locations are in native script remain the hardest at 0.787. Overall, the TEST AUC is 0.913 with a Brier score of 0.120.

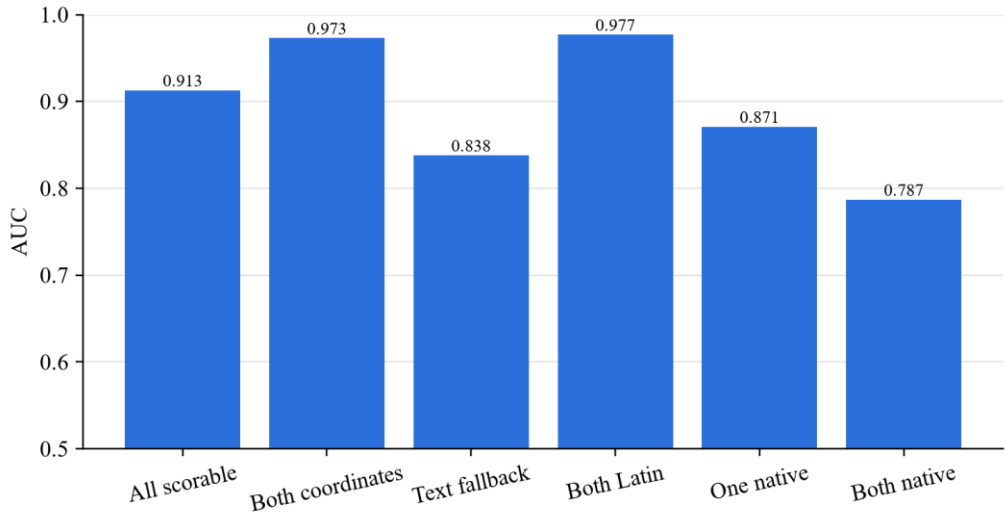


Figure 6.2: Step 3B TEST AUC by kind of report pair

Geography alone cannot separate every hard case. On the report pairs, two reports at the same place on different days scored 0.68 on average and two adjacent houses scored 0.28; these cases must be separated by time and by the evidence judge.

6.1.4 Step 4: time

The per-type spline model was chosen over a single model for all types on DEV (log loss 0.0535 against 0.0593). On TEST the time score reached an AUC of 0.996 in the arrival-order replay, and recall of the true link was 0.947 at rank 1 and 1.0 at rank 3 when candidates were ranked by geography and time together. On the report pairs, two reports at the same place on different days, which geography had scored at 0.68, received a mean time score of 0.002, showing that the two channels complement each other. Figure 6.3 shows the selection of the candidate cap K on the stress replay; with K = 23 every true DEV link was kept, and on the TEST stress replay 98.7% were kept.

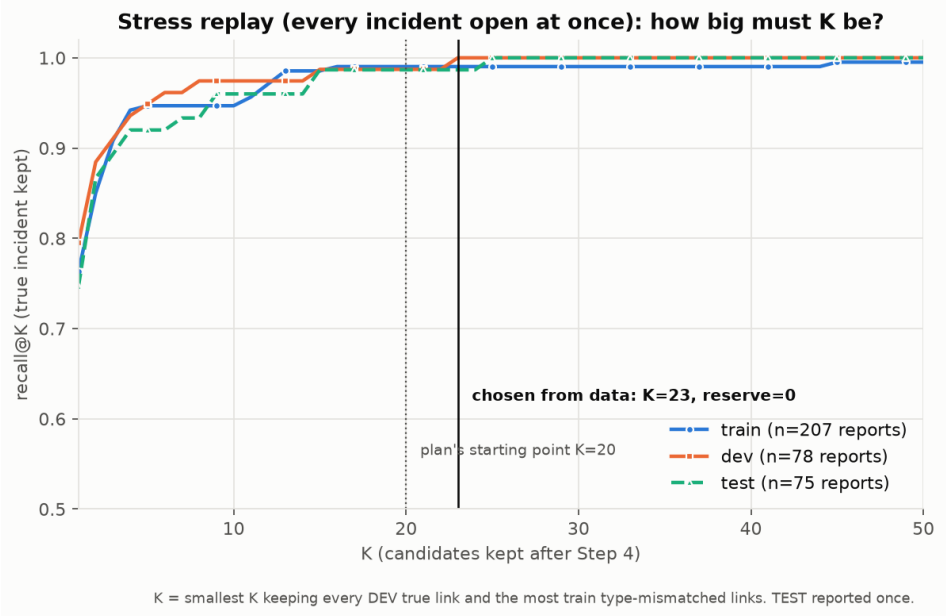


Figure 6.3: Recall of true links as a function of the candidate cap K on the stress replay

6.1.5 Step 5A: semantic similarity

The fine-tuned LaBSE encoder passed all seven health checks on Colab (loss from 2.957 to 0.001; three seeds 0.972, 0.972 and 0.970) and all thirteen laptop verification checks. Figure 6.4 compares it with the baselines. On TEST the fine-tuned model reached an AUC of 0.946 against 0.846 for the off-the-shelf model and 0.621 for character TF-IDF. Removing the positive pairs that are near-translations of each other changed the TEST AUC only from 0.946 to 0.941, so the gain is not due to translation pairs alone. By language, the AUC was 0.972 for pairs with Singlish, 0.903 for pairs with Tanglish and 0.941 for native or mixed pairs (Figure 6.5).

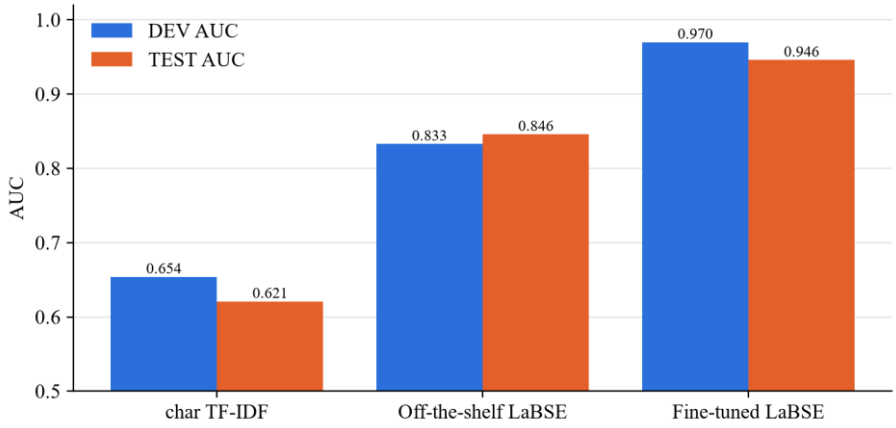


Figure 6.4: AUC of semantic similarity on the report pairs: baselines and the fine-tuned encoder

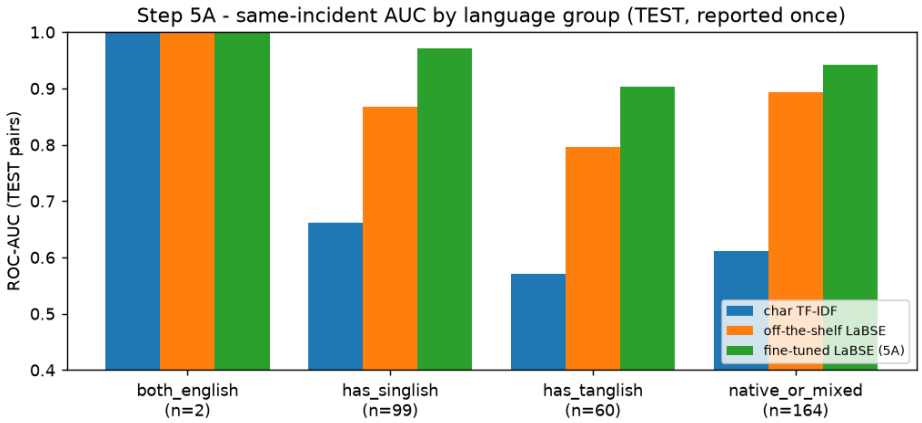


Figure 6.5: TEST AUC of semantic similarity by language of the report pair

Semantic similarity has a predictable weakness: two reports at the same place on different days use the same words, and their mean cosine rose with fine-tuning to 0.80. Adjacent houses with separate dispatches stayed at 0.50. Both cases have to be resolved by the other channels.

6.1.6 Step 5B: evidence judge

The evidence judge was trained in three runs. The first run was rejected by its health report because its three seeds disagreed (macro-F1 0.709, 0.820 and 0.741); the analysis showed that the grid search had used a single seed and a different learning-rate schedule from the final model. The training procedure was corrected so that every configuration is scored with all three seeds and the kept model is the snapshot saved at the selected epoch. The second run passed all nine checks and reached a TEST macro-F1 of 0.718 (Figure 6.6). Per class, the F1 was 0.75 for corroborates, 0.81 for contradicts, 0.65 for neutral and 0.67 for count_variance.

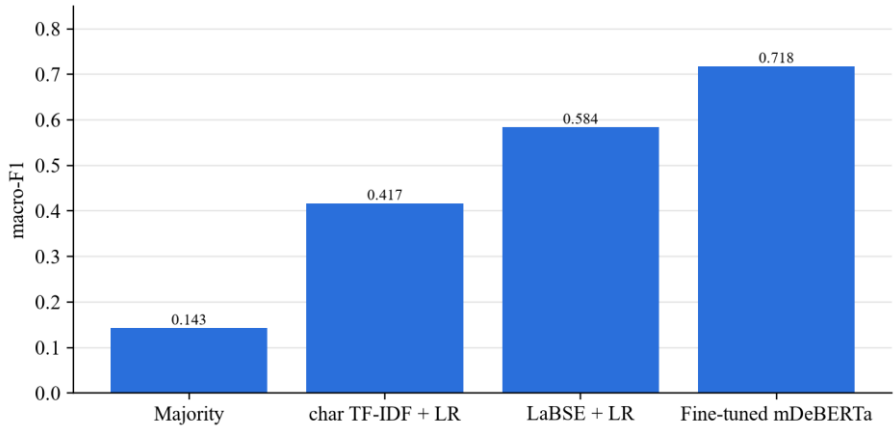


Figure 6.6: TEST macro-F1 of the evidence judge against baselines (four labels)

In live testing the second run labelled '23 පන්සල පාර' against '14 පන්සල පාර' as count_variance with probability 0.93, reading the house number as a count. The corpus contained almost no real pairs of this kind, so 156 synthetic pairs were added in which only the house number of a real caller answer was changed (dataset v6). An adoption rule with four conditions was written before the third run: health and laptop verification must pass, DEV macro-F1 on the real rows must not fall by more than 0.01, accuracy on the synthetic DEV rows must improve, and the DEV AUC of the judge features on the report pairs must not fall by more than 0.005. Table 6.3 shows that all four conditions were met, and the third run was adopted.

Table 6.3: Adoption rule for the third evidence judge run (all values on DEV)

Condition	Run 2	Run 3	Met
Health report and laptop verification	-	9/9, verified	Yes
Macro-F1 on real rows >= 0.7955	0.8055	0.8299	Yes
Accuracy on synthetic rows higher than run 2	25/29	29/29	Yes
Report-pair AUC of judge features >= 0.9233	0.9283	0.9481	Yes

TEST, reported once after adoption, disagreed with DEV: the macro-F1 on the real rows fell from 0.718 to 0.687 and the report-pair AUC of the judge features fell from 0.915 to 0.886, while TEST same-incident pairs received more contradictions than on DEV. This DEV/TEST disagreement is discussed in Section 7.5. On the report pairs, the judge added to semantic similarity on DEV (cross-validated AUC 0.985 for both against 0.971 for semantic similarity alone) and on TEST with the second run (0.953 against 0.946). Contradictions were concentrated where they matter: 58% of

adjacent-house trap pairs received a contradiction, against 2 to 5% of same-incident pairs and none of the easy negatives.

A further retraining with spelling-variant and identity pairs (dataset v7) improved the judge on its own DEV measures but made the fusion model worse end to end (false merges 0.9% to 3.6% on the DEV replay), and was not adopted after a stability check over ten fold divisions.

6.1.7 Step 6: fusion

Fitting the fusion model on TRAIN was rejected because every channel model was trained on TRAIN and its scores there are in-sample; for example the semantic AUC is 1.000 on TRAIN, 0.970 on DEV and 0.946 on TEST. Among the models fitted on DEV, gradient boosting had the lowest cross-validated log loss (0.069 against 0.079 for logistic regression, within the noise). The report-level fusion model reached a TEST log loss of 0.175, a Brier score of 0.051, an AUC of 0.983 and an expected calibration error of 0.033. Figure 6.7 shows its reliability diagram.

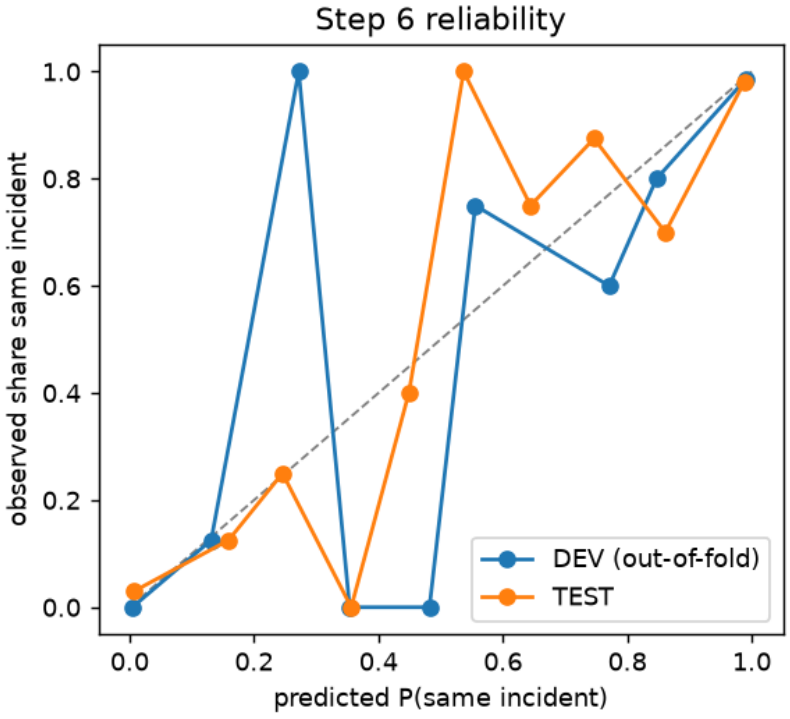


Figure 6.7: Reliability diagram of the fusion probability

The ablation in Figure 6.8 removes one channel at a time and refits the model. Removing time hurts most (TEST log loss 0.388), followed by semantic similarity

(0.232) and the evidence judge (0.202); removing geography costs less (0.186), because time and semantic similarity already carry much of the place information in this corpus, and the type features add nothing beyond the per-type time models. Every scoring channel therefore contributes, and in particular the evidence judge earns its place, which was the critical comparison planned for this component.

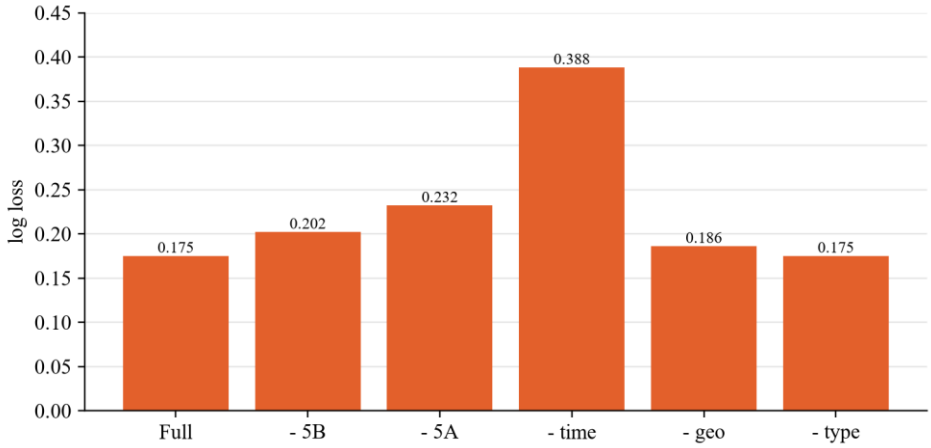


Figure 6.8: Channel ablation of the fusion model (TEST log loss when each channel is removed)

6.1.8 Step 7: cost-based decision

Figure 6.9 shows how the error rates change with the cost ratio on the DEV replay. Raising the threshold lowers false merges slowly but raises false splits and the review load quickly. The fusion model trained on real data alone could not bring false merges below 7% at any ratio, because three adjacent-house traps always scored high; adding the synthetic adjacent-house reports halved the false merges and removed the trap merges.

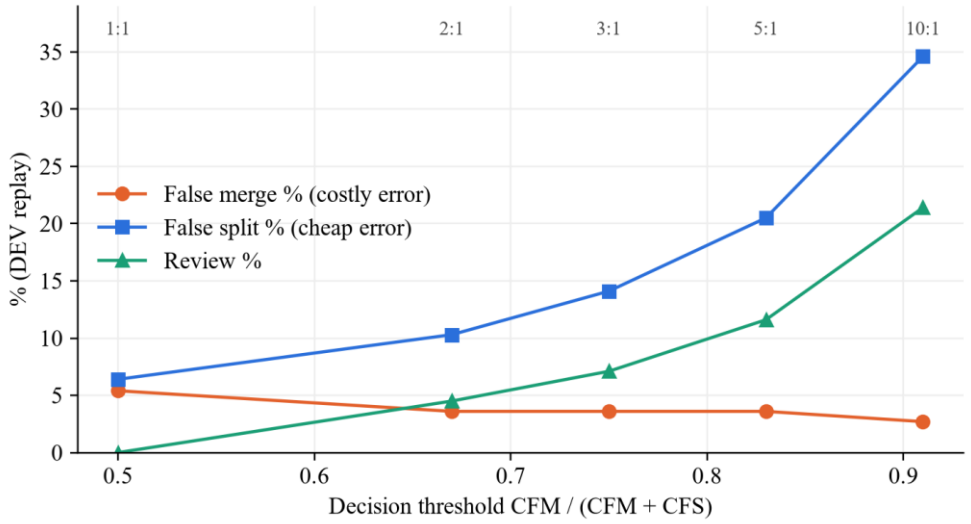


Figure 6.9: Error rates as a function of the cost ratio (DEV replay, report-level matching, augmented fusion model)

At report level, with a 2:1 cost ratio, TEST gave a false merge rate of 0.9% and a merge precision of 98.2%, but a false split rate of 24.0%: C4 erred strongly towards splitting. A diagnosis on DEV showed that the synthetic negatives had taught the model that near-identical text with many corroborations is suspicious. Several fixes were tried on DEV against a rule written beforehand and none passed; the fix that worked was to change the unit of comparison.

6.1.9 Step 8: incident-level matching

Comparing each report with the whole incident instead of single earlier reports passed its DEV adoption rule and was adopted, with a 3:1 cost ratio chosen by the researcher. Figure 6.10 compares the two on TEST: the false split rate fell from 24.0% to 10.7% and link recall rose from 74.7% to 88.0%, with false merges at 1.7% (two reports, both adjacent-house traps) and a B-cubed precision and recall of 0.980 and 0.897. This answers RQ3: pooling the evidence of an incident substantially reduces false splits.

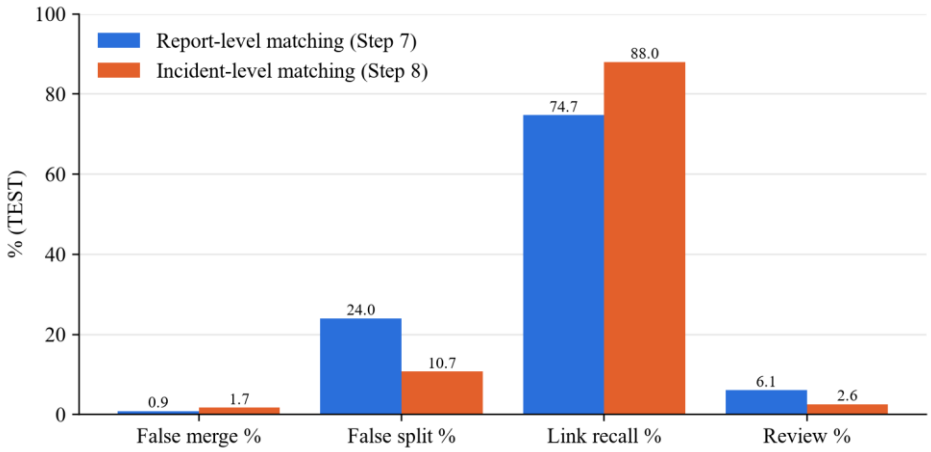


Figure 6.10: Report-level against incident-level matching on TEST

Two further changes to the fusion model were made after live cases. Counterfactual type rows removed every fire-flood merge on DEV without a hand-written rule; flipped same-incident pairs scoring above the threshold fell from 66 of 72 to 0 of 72. House-number features with counterfactual rows removed the remaining live merges of different house numbers on the same road and, on the original DEV fold division, lowered false merges from 1.8% to 0.9% and false splits from 17.9% to 16.7%.

6.1.10 Step 9: revision and the ambiguity flag

Table 6.4 shows the DEV replay of the revision variants. Retrospective merging of two incidents when a report scored above the threshold for both made four merges, three of them wrong (one an adjacent-house trap), and lowered the B-cubed F1; detaching never fired. Neither was adopted. Using the same signal to ask instead of act was adopted: a report above the threshold for two or more incidents is flagged for review. On TEST, 6 of 115 reports were flagged (5.2%), 4 of the 6 flags pointed at a real problem, and one of the two wrong merges was caught.

Table 6.4: Revision variants on the DEV replay

Variant	B-cubed P / R / F1	Wrongly grouped %	Retro merges
None (adopted, with ambiguity flag)	0.972 / 0.877 / 0.922	8.0	0
Retrospective merge	0.917 / 0.897 / 0.907	17.0	4 (1 right)
Detach	same as none	8.0	0

6.2 Product and Technical Results

Table 6.5 gives the final results of the component as it runs live: mT5-small student, learned type reader, incident-level fusion with synthetic and counterfactual rows, the no-location rule, the ambiguity flag and a 3:1 cost ratio. Because the selection had used one fixed fold division of DEV, and that division turned out to be the most favourable of ten, the DEV numbers are given as the mean and standard deviation over ten fold divisions. TEST was scored once for this configuration, but had been scored for earlier configurations before; nothing was changed after this run. Figure 6.11 shows the same numbers.

Table 6.5: Final results of the live configuration (DEV: mean (sd) over ten fold divisions; TEST: once)

Measure	DEV live	DEV without rules	TEST live	TEST without rules
False merge %	2.43 (0.70)	2.34 (0.83)	0.9	1.7
False split %	19.73 (1.54)	12.04 (1.53)	28.0	14.7
Link recall %	77.81 (1.84)	85.50 (1.82)	70.7	84.0
Merge precision %	95.74 (1.24)	96.25 (1.31)	98.1	96.9
Review %	8.55 (1.84)	8.55 (1.96)	6.1	6.1
Adjacent-trap merges	1.0 (0.78)	1.0 (0.78)	1	2
B-cubed P / R / F1	0.968 / 0.793 / 0.872	0.969 / 0.851 / 0.906	0.991 / 0.765 / 0.864	0.980 / 0.868 / 0.921
Incidents built (gold)	48.6 (34)	42.7 (34)	61 (40)	50 (40)

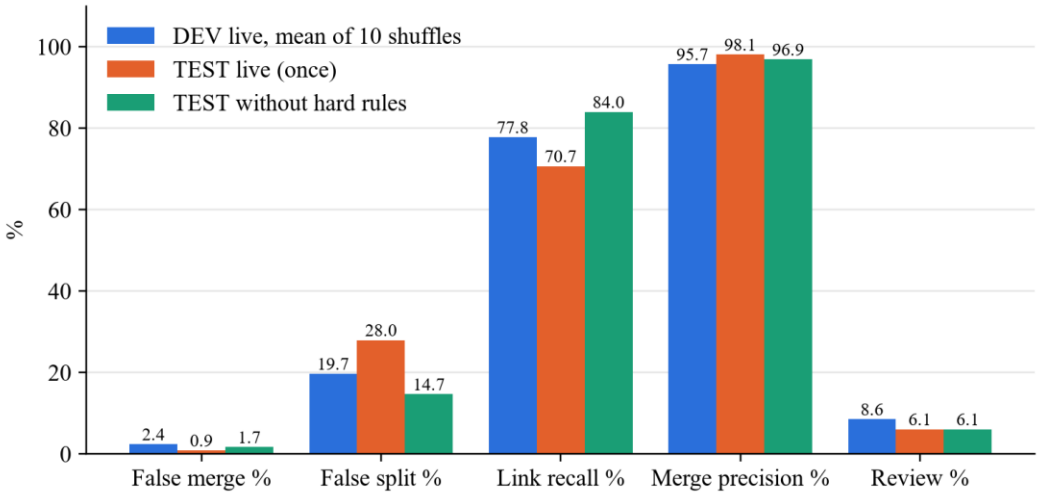


Figure 6.11: Final operational results: DEV (mean of ten fold divisions) and TEST

The live configuration keeps false merges at 0.9% of TEST reports with a merge precision of 98.1%, which meets the safety requirement NFR2. The price is a high false split rate. The 'without rules' columns show that most of this price comes from

the researcher's no-location rule: TEST contains twelve reports without any location, and the rule keeps each of them, and anything that follows them, apart. Without the rule, link recall on TEST is 84.0% at a false merge rate of 1.7%.

In terms of speed, the reply to C2 and C3 takes about four seconds on the laptop CPU, and the background steps finish within a few seconds after it, well inside the 20-second budget (NFR1). In terms of robustness, a run with the extraction model switched off reproduced the outage path: the recovered and upstream values carried on and the geography step continued to work (NFR4).

6.3 User or Stakeholder Validation

A formal study with dispatchers or emergency call-takers was not carried out within the project period, and no claim is made about how dispatchers would use the dashboard. Validation of the user-facing parts was limited to the following. The operator page and the dashboard were used by the author throughout development to examine live reports entered through C2 and C3 by the group, and each live fault found in this way was turned into a test and a measured fix (Table 5.2). The dashboard design was reviewed for legibility: its colour palettes were checked for contrast in light and dark themes, low-contrast colours always carry text labels, and the layout was checked at phone width. A study with real dispatchers on real or replayed hotline traffic is the most important piece of future work (Section 9.4).

6.4 Integration Results

The integration with C1, C2 and C3 was verified at three levels. The contract tests confirm that a valid report is accepted and that a report with an unknown or missing field is rejected with an explanatory error. The integration check script started a real C4 server and published C2 and C3 sessions through the real shared client; both channels were received, processed and stored. In end-to-end live tests the components ran together and reports from both channels were fused into common incidents; in one recorded session, for example, 23 reports formed 17 incidents, five of which had been reached by both calls and SMS (Figure 5.2).

Live testing also showed the effect of upstream errors on fusion. Test SMS messages were sometimes typed as non-emergency or with the wrong type by the extractor,

and an operator's menu answer could be recorded as the caller's statement. In each case the reconciled record showed the disagreement between the sources, and the fixes listed in Section 5.4 were measured on DEV before adoption.

6.5 Summary of Findings

- Learned cross-script place comparison works where geocoding does not: in text fallback the geography score reached a TEST AUC of 0.838, and Places itself was reliable only for Latin script (RQ1).
- The evidence judge adds information beyond overall similarity, mainly by finding contradictions in the hard negatives (58% of adjacent-house traps), but its benefit on TEST was smaller than on DEV (RQ2).
- Matching against the whole incident reduced the TEST false split rate from 24.0% to 10.7% and raised link recall from 74.7% to 88.0% (RQ3).
- A 3:1 cost ratio keeps false merges near 1% of reports, but at the cost of false splits that are sensitive to the data split; DEV numbers from a single fold division were optimistic, and multi-seed reporting is needed (RQ4).
- The component meets its timing and robustness requirements and is integrated live with the other three components.

7 DISCUSSION

7.1 Interpretation of Findings

The results support the main design idea of the component: no single channel is enough, but the channels fail in different places. Geography cannot tell a report from last week at the same place from a report from today; time can. Semantic similarity rises when two callers use the same words, even about different houses; the evidence judge finds the conflicting house number. Time cannot tell two adjacent houses apart, because both are reported at the same moment; geography with house-number features and the evidence judge can. The ablation confirms that removing any of the four scoring channels makes the fusion probability worse.

The most important single finding is the effect of incident-level matching. At report level, a new report is compared with each earlier report separately, and the evidence that links it to its incident may be spread across several of them: one earlier report gives the address, another the number of people trapped. Comparing with the pooled evidence of the incident gives the fusion model the whole picture, and the false split rate on TEST fell by more than half. This agrees with the experience of topic detection and tracking, where comparing a story with a cluster was more reliable than comparing it with single stories [6].

The second finding concerns the cost of safety. A cost ratio of 3:1 and the researcher's no-location rule keep false merges at about one percent, but they move errors into false splits and review. This is the intended direction, because a false split leaves the emergency visible while a false merge can hide it. The size of the effect, however, depends strongly on how many reports lack a location: TEST has twelve such reports and DEV seven, and the no-location rule alone accounts for much of the difference between the DEV and TEST false split rates. The decision between the rule and the learned model is a policy decision that should be made by the emergency service with these numbers in front of it, and the component makes it a configuration switch for that reason.

The third finding is methodological. Several numbers that looked good on a single DEV fold division turned out to be optimistic when the division was varied: the fold

division used for most selections was the most favourable of ten, with a false merge rate of 0.9% against a mean of 2.4%. With small evaluation sets, a single split can flatter a model by chance. Reporting the mean and spread over several divisions, and writing the adoption rule before the run, were the two practices that made the results trustworthy.

7.2 Comparison with Literature Baselines and Existing Solutions

Direct comparison with published systems is not possible, because no published dataset covers multilingual emergency reports in Sinhala and Tamil and no published system addresses the same task. The comparisons that can be made are with the baselines that represent the methods of the literature, and these were built and measured on the same data. For semantic similarity, the fine-tuned LaBSE encoder improved the TEST AUC from 0.846 for the off-the-shelf model [22] and 0.621 for character TF-IDF to 0.946. For evidence matching, the fine-tuned mDeBERTa judge reached a TEST macro-F1 of 0.718 against 0.584 for a frozen LaBSE classifier and 0.417 for TF-IDF; for reference, models of this family reach much higher scores on standard NLI benchmarks in high-resource languages [25], [24], which shows how much harder short, code-mixed answers in Sinhala and Tamil are. For place comparison, the learned model outperformed hand-written phonetic rules of the kind used in classical record linkage [3] (DEV macro-F1 0.777 against 0.720).

Compared with the duplicate-call features of commercial dispatch systems, which rely on a geocoded address and a human decision, the component offers three things those systems do not: it works without a geocoded address by comparing place descriptions across scripts, it compares what callers actually said, and it gives a calibrated probability and an evidence trail rather than a list of nearby calls. Its B-cubed F1 on TEST (0.864 live, 0.921 without the no-location rule) is not directly comparable with the scores reported for cross-document event coreference on ECB+ [4], [5], because those systems cluster mentions in news text off-line, whereas the component must decide on-line, in arrival order, with no access to later reports.

7.3 Research Contribution

- A multilingual evidence-level judge for emergency reports with a four-way label set that separates count differences from contradictions, trained and evaluated on Sinhala, Tamil, English and code-mixed answers, together with evidence that synthetic minimal-change pairs teach it to read house numbers as place identity.
- A rule-free, cross-script place comparison model that reads native script through transliteration and is pre-trained on gazetteer pairs, and a measurement showing that a commercial geocoder is unreliable for Sinhala and Tamil script, so that text comparison is preferable to a confident wrong coordinate.
- Evidence that incident-level matching against pooled evidence substantially reduces false splits in on-line report fusion.
- The use of counterfactual rows to teach a fusion model hard constraints (different incident types, different house numbers) without hand-written rules, with a flip test that verifies the constraint.
- An evaluation protocol for small, safety-relevant datasets: incident-level splits with trap groups, replay of the live code path, adoption rules written before the run, and multi-seed reporting.

7.4 Product Contribution

As a product, the component turns the separate outputs of the voice and SMS channels into one operational picture. A dispatcher sees one incident per emergency rather than one line per report, with the fields consolidated from all reports, a timeline of what each report added and a map. Each merge can be explained by its evidence trail, and every uncertain or ambiguous case is visible as a review flag next to the incident it may match. The component runs on an ordinary laptop without a GPU, works without any paid API at run time in its default evaluated configuration, and degrades gracefully when a model or service is unavailable.

7.5 Limitations and Threats to Validity

- Simulated data. The corpus is purpose-built and simulated. Real calls contain more noise, longer silences, speech recognition errors and a different mix of incidents. The urgency component of the group showed a large drop when it was

validated on real recordings, and a similar drop is likely here. All results should be read as development estimates, not as field performance.

- Label quality. The reference labels were drafted with a large language model and checked by a person on a sample of forty reports and forty evidence pairs (agreement 89.4% and 77.5%). The check did not cover Tamil or Tanglish, and remaining label noise limits the accuracy that can be measured.
- Small evaluation sets. DEV and TEST contain 112 and 115 reports, of which 78 and 75 are linkable. One report changes a rate by about one percentage point, and the hard cases (adjacent-house traps) number only a few per split.
- DEV/TEST disagreement. The third evidence judge improved every DEV measure but was worse on TEST, and TEST same-incident pairs received more contradictions than DEV pairs. With small data this may be chance or a real difference between the splits; it was reported and not used to change the model.
- Test re-use. TEST was scored more than once during the project as the configuration evolved. Choices were always made on DEV and nothing was changed after a TEST run, but repeated looks at TEST weaken its independence; this is disclosed with each result.
- Policy choices. The no-location rule and the house-number counterfactual rows were kept by the researcher's decision; the latter did not meet its own multi-seed adoption rule by a small margin (adjacent-trap merges 1.0 against 0.9). They are reported as policy, not as measured gains.
- Template questions. The operator questions in the corpus come from a fixed set of templates, which makes question classification easier than it would be with free-form human operators.
- Deployment configuration. The evaluation was done with Places limited to its cache and with native-script locations kept as text. A deployment can switch on live lookups and native-script lookups; that configuration was not evaluated and, by the Places measurement, may introduce wrong coordinates for Sinhala and Tamil locations.

7.6 Implications

For emergency services, the results suggest that automatic fusion of reports is feasible for multilingual voice and SMS channels, but that it should be deployed as decision support with review, not as an autonomous system. The trade-off between false merges and false splits is a policy question, and the component exposes it through a cost ratio and named rules whose effect has been measured. The finding about geocoding implies that services should not trust coordinates obtained from native-script place descriptions without verification.

For research, the work shows that small, carefully designed datasets with explicit hard cases can support useful models in low-resource languages, provided that evaluation is strict. It also shows that synthetic minimal-change examples are an effective way to teach models about rare but decisive cases, but that their side effects must be measured end to end: synthetic negatives that fixed the adjacent-house trap also raised false splits, and only a change of matching unit recovered them.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The fusion component offers emergency and disaster response organisations a single, explained view of each emergency built from all the calls and messages that report it, in the languages that people in Sri Lanka actually use. Its value lies in three outcomes: fewer duplicate dispatches during peaks of report volume, fewer emergencies lost among repeat reports, and an audit trail that shows why reports were grouped.

The primary target users are national and provincial emergency and disaster call centres, such as the Disaster Management Centre's hotline, police and fire emergency services, and provincial emergency operation centres that coordinate flood response. Secondary users are hospital and ambulance dispatch services, non-governmental relief organisations that run their own reporting lines during disasters, and municipal councils that receive complaints about local hazards. The direct users of the dashboard are call-takers, dispatchers and shift supervisors.

8.2 Market and Competitive Position

Computer-aided dispatch systems are an established market dominated by large international vendors whose products are designed for single-language, address-based operation and are priced for well-funded services. Their duplicate-call support is based on address proximity and the call-taker's judgement. In Sri Lanka and in neighbouring countries with similar multilingual populations, many reporting lines still record reports manually or in general-purpose software.

The component's competitive position rests on three advantages: native support for Sinhala, Tamil and their code-mixed romanised forms; comparison of what callers said rather than only where they are; and low running cost, since inference runs on a CPU without a paid API. It is best positioned not as a replacement for a dispatch system but as a fusion and decision-support module that can be connected to an existing intake system through its API, as it is connected to C2 and C3 in this project.

8.3 Business or Sustainability Model

Because the main customers are public bodies, a pure commercial licence is unlikely to be the most suitable model. Three models are possible and can be combined.

- Government service contract. The component, together with the other components of the group's solution, is offered to a public emergency service under a contract that covers installation, adaptation to the service's own data, training of staff and annual maintenance.
- Open core with paid support. The core code is released under an open-source licence so that public services and researchers can inspect it, which suits a safety-relevant system, while adaptation, model retraining on a customer's data, hosting and support are offered as paid services.
- Donor-funded deployment. Disaster-risk-reduction programmes funded by international development partners often finance early-warning and response tools; a pilot deployment for flood-prone districts could be funded this way.

Operating costs are low. The models run on a standard server CPU, so a single server can serve a call centre. External services are optional: geocoding lookups and language-model lookups are billed per request but are cached, so each distinct place string is paid for once, and both can be switched off entirely. The main ongoing cost is the periodic retraining of the models on the customer's own reports, which requires labelled data and a GPU for a few hours.

8.4 Scalability Intellectual Property and Compliance

The current implementation processes reports one at a time with a single background worker, which is sufficient for a single call centre. Scaling to a national service would require a shared database for the incident store instead of local files, several workers with a lock per region so that reports about one area are still processed in arrival order, and a limit on the number of open incidents compared, which the candidate cap already provides. Because each scoring channel is independent, channels can run in parallel.

The intellectual property of the component consists of its code, its trained models and its dataset. The models are fine-tuned from openly licensed pre-trained models

(mT5, LaBSE and mDeBERTa), whose licences permit commercial use with attribution, and the gazetteer data are licensed under CC-BY 4.0. The use of the teacher model's outputs to train the student must comply with the provider's terms of use. In a real deployment, reports contain personal data, so the system must comply with Sri Lanka's Personal Data Protection Act, No. 9 of 2022: data minimisation, retention limits, access control and a clear legal basis for processing. Sending caller text to external services should be disabled or covered by an agreement with the provider, and the component already supports running without any external call.

8.5 Contribution to the Sustainable Development Goals

- SDG 11, Sustainable cities and communities, target 11.5: reducing the number of deaths and people affected by disasters, including water-related disasters. Faster and more accurate grouping of flood reports helps responders reach every distinct emergency.
- SDG 13, Climate action, target 13.1: strengthening resilience and adaptive capacity to climate-related hazards and natural disasters. Floods are among the most frequent climate-related hazards in Sri Lanka.
- SDG 3, Good health and well-being: timely emergency response reduces avoidable deaths and injuries.
- SDG 10, Reduced inequalities: supporting Sinhala, Tamil and code-mixed reports equally means that people are not disadvantaged by the language in which they ask for help.
- SDG 9, Industry, innovation and infrastructure: the component contributes locally developed AI infrastructure for public services in low-resource languages.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the multi-source incident fusion component (C4) of a multilingual emergency intake and triage pipeline for Sri Lanka. For every call or SMS report that reaches it, the component decides whether the report belongs to an emergency that is already open or describes a new one. It does so by reconciling the report into a reliable record while keeping the caller's words verbatim, scoring it against the open incidents on four independent learned channels (geography, time, overall meaning and fine-grained evidence), fusing them into a calibrated probability, and deciding with a threshold derived from the asymmetric cost of the two possible errors. The result is kept as a fused incident record with an evidence trail and is shown to the dispatcher on a live dashboard.

Evaluated on a purpose-built multilingual corpus of 535 reports with development-only selection, the component in its live configuration merged wrongly only 0.9% of TEST reports, with a merge precision of 98.1%, while linking 70.7% of repeat reports; without the conservative no-location rule it linked 84.0% at a false merge rate of 1.7%. Matching against whole incidents instead of single reports was the most effective single design choice, more than halving the false split rate. The work also showed that a commercial geocoder is unreliable on Sinhala and Tamil script, that a learned model can compare place descriptions across scripts instead, and that an evidence judge trained to separate count differences from contradictions catches many of the hardest cases, two houses on the same road.

9.2 Achievement of Objectives

Table 9.1: Achievement of the objectives

Objective	Outcome	Evidence
O1 Reliable structured record without altering the caller's words	Achieved	Step 1e accuracy DEV 95-100%, TEST 91-100% per field; evidence text identical to caller words (tested)
O2 Independent multilingual scoring channels	Achieved	TEST AUC: geography 0.913, time 0.996 (replay), semantic 0.946; judge macro-F1 0.718
O3 Calibrated fusion with ablation	Achieved	TEST AUC 0.983, ECE 0.033; every channel contributes in the ablation

Objective	Outcome	Evidence
O4 Cost-aware decision with review	Achieved	3:1 threshold; TEST false merge 0.9%, review 6.1%; ambiguity flag 4 of 6 useful
O5 Honest evaluation and live integration	Achieved, with limits	DEV-only selection, multi-seed DEV, TEST once per model; live with C1-C3; no user study

9.3 Individual Research and Product Contributions

The individual research contributions are a multilingual four-way evidence judge for emergency reports, a rule-free cross-script place comparison model with a measurement of geocoding reliability by script, evidence for the value of incident-level matching in on-line report fusion, the use of counterfactual rows to teach a fusion model hard constraints without rules, and an evaluation protocol for small safety-relevant datasets. The product contributions are the working fusion service, its integration with the other three components through a versioned contract, the operator testing page and the incident dashboard, and the versioned dataset with its trap groups, labelled pairs and evidence pairs.

9.4 Future Directions

- Validation on real reports. The most important next step is to evaluate the component on real, anonymised hotline calls and SMS reports, for example a set of incident groups from the Disaster Management Centre held back as an untouched external test set, and to retrain on such data where permitted.
- User study. A study with call-takers and dispatchers should measure whether the dashboard and the review flags help them make faster and better decisions, and how they would set the cost ratio and the policy rules.
- Better data for the hard cases. More real examples of adjacent houses, of reports without a location and of same-incident reports in different scripts would allow the evidence judge and the fusion model to learn these cases from real rather than synthetic data, and would make the DEV/TEST comparison more stable.
- Geocoding for native script. Transliteration before geocoding, or a gazetteer of Sri Lankan places with native-script names, could make coordinates available for Sinhala and Tamil reports without the errors measured here.

- Speech recognition errors. When C2 runs on real audio, its transcripts will contain recognition errors; the channels should be retrained on such transcripts and the component should use the recogniser's confidence.
- Scaling. A shared database, regional workers and monitoring would be needed for a national deployment.

REFERENCES

[1] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," Journal of the American Statistical Association, vol. 64, no. 328, pp. 1183-1210, 1969.

[2] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios, "Duplicate record detection: A survey," IEEE Transactions on Knowledge and Data Engineering, vol. 19, no. 1, pp. 1-16, 2007.

[3] P. Christen, Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Berlin, Germany: Springer, 2012.

[4] A. Cybulska and P. Vossen, "Using a sledgehammer to crack a nut? Lexical diversity and event coreference resolution," in Proc. 9th Int. Conf. Language Resources and Evaluation (LREC), Reykjavik, Iceland, 2014, pp. 4545-4552.

[5] K. Lee, L. He, M. Lewis, and L. Zettlemoyer, "End-to-end neural coreference resolution," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), Copenhagen, Denmark, 2017, pp. 188-197.

[6] J. Allan, J. Carbonell, G. Doddington, J. Yamron, and Y. Yang, "Topic detection and tracking pilot study: Final report," in Proc. DARPA Broadcast News Transcription and Understanding Workshop, 1998, pp. 194-218.

[7] M. Imran, C. Castillo, F. Diaz, and S. Vieweg, "Processing social media messages in mass emergency: A survey," ACM Computing Surveys, vol. 47, no. 4, pp. 1-38, 2015.

[8] A. Olteanu, C. Castillo, F. Diaz, and S. Vieweg, "CrisisLex: A lexicon for collecting and filtering microblogged communications in crises," in Proc. 8th Int. AAAI Conf. Weblogs and Social Media (ICWSM), 2014, pp. 376-385.

[9] S. Vieweg, A. L. Hughes, K. Starbird, and L. Palen, "Microblogging during two natural hazards events: What Twitter may contribute to situational awareness," in Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI), 2010, pp. 1079-1088.

[10] N. de Silva, "Survey on publicly available Sinhala natural language processing tools and research," arXiv preprint arXiv:1906.02358, 2019.

[11] B. R. Chakravarthi, V. Muralidaran, R. Priyadharshini, and J. P. McCrae, "Corpus creation for sentiment analysis in code-mixed Tamil-English text," in Proc. 1st Joint Workshop on Spoken Language Technologies for Under-resourced Languages (SLTU) and Collaboration and Computing for Under-Resourced Languages (CCURL), 2020, pp. 202-210.

[12] International Academies of Emergency Dispatch, "Medical Priority Dispatch System (MPDS)," Salt Lake City, UT, USA. [Online]. Available: https://www.emergencydispatch.org

[13] C. Elkan, "The foundations of cost-sensitive learning," in Proc. 17th Int. Joint Conf. Artificial Intelligence (IJCAI), Seattle, WA, USA, 2001, pp. 973-978.

[14] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in Advances in Large Margin Classifiers, A. J. Smola, P. Bartlett, B. Scholkopf, and D. Schuurmans, Eds. Cambridge, MA, USA: MIT Press, 1999, pp. 61-74.

[15] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in Proc. 8th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2002, pp. 694-699.

[16] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. 34th Int. Conf. Machine Learning (ICML), Sydney, Australia, 2017, pp. 1321-1330.

[17] G. W. Brier, "Verification of forecasts expressed in terms of probability," Monthly Weather Review, vol. 78, no. 1, pp. 1-3, 1950.

[18] M. Imran, P. Mitra, and C. Castillo, "Twitter as a lifeline: Human-annotated Twitter corpora for NLP of crisis-related messages," in Proc. 10th Int. Conf. Language Resources and Evaluation (LREC), Portoroz, Slovenia, 2016, pp. 1638-1643.

[19] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzman, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 8440-8451.

[20] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, 2019, pp. 3982-3992.

[21] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 4512-4525.

[22] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, and W. Wang, "Language-agnostic BERT sentence embedding," in Proc. 60th Annual Meeting of the Association for Computational Linguistics (ACL), Dublin, Ireland, 2022, pp. 878-891.

[23] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," in Proc. Conf. Empirical

Methods in Natural Language Processing (EMNLP), Lisbon, Portugal, 2015, pp. 632-642.

[24] A. Conneau, R. Rinott, G. Lample, A. Williams, S. Bowman, H. Schwenk, and V. Stoyanov, "XNLI: Evaluating cross-lingual sentence representations," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), Brussels, Belgium, 2018, pp. 2475-2485.

[25] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in Proc. 11th Int. Conf. Learning Representations (ICLR), Kigali, Rwanda, 2023.

[26] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531, 2015.

[27] L. Xue, N. Constant, A. Roberts, M. Kale, R. Al-Rfou, A. Siddhant, A. Barua, and C. Raffel, "mT5: A massively multilingual pre-trained text-to-text transformer," in Proc. Conf. North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2021, pp. 483-498.

[28] D. Kaushik, E. Hovy, and Z. C. Lipton, "Learning the difference that makes a difference with counterfactually-augmented data," in Proc. 8th Int. Conf. Learning Representations (ICLR), 2020.

[29] Google, "Places API documentation," Google Maps Platform. [Online]. Available: https://developers.google.com/maps/documentation/places/web-service

[30] GeoNames, "GeoNames geographical database (country extract LK)," licensed under CC-BY 4.0. [Online]. Available: https://www.geonames.org

[31] A. Bagga and B. Baldwin, "Algorithms for scoring coreference chains," in Proc. 1st Int. Conf. Language Resources and Evaluation (LREC) Workshop on Linguistic Coreference, Granada, Spain, 1998, pp. 563-566.

[32] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[33] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems 30 (NIPS), 2017, pp. 3146-3154.

[34] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP), 2020, pp. 38-45.

GLOSSARY

Adjacent-house trap - Two different incidents at neighbouring addresses on the same road, reported at about the same time, which need separate dispatches.

Code-mixing - Use of two or more languages within one utterance, for example Sinhala words with English place names.

Counterfactual row - A copy of a real training example in which only the features of interest are changed, labelled with the outcome that change should cause.

False merge - A report attached to an incident it does not belong to.

False split - A report that belongs to an open incident but is recorded as a new incident.

Incident - One real-world emergency, which may be reported by several calls and messages.

Singlish / Tanglish - Sinhala or Tamil written in Latin letters, often mixed with English.

Text fallback - Geography mode in which a location is compared as text by the learned place model because no reliable coordinates are available.

Trap group - A group of different incidents that are easy to confuse because they share a place, road, type or day.

APPENDICES

Appendix A Extended Results

Table A.1 gives the Step 8 DEV replay that led to the adoption of incident-level matching, and Table A.2 the multi-seed re-check of the two fusion design decisions taken after live testing.

Table A.1: Step 8 DEV replay: report-level against incident-level matching (single fold division)

Configuration	FM %	FS %	Recall %	Review %	Trap merges	B-cubed P / R
Report level, augmented, 2:1	3.6	10.3	87.2	4.5	0	-
Incident level, augmented, 2:1	2.7	9.0	89.7	0.0	0	0.951 / 0.874
Incident level, augmented, 3:1 (adopted)	1.8	9.0	89.7	1.8	0	0.972 / 0.877
Incident level, real only, 2:1	3.6	7.7	91.0	1.8	1	0.928 / 0.892

Table A.2: Multi-seed re-check of the fusion decisions (DEV, mean (sd) over ten fold divisions)

Configuration	FM %	FS %	Trap	FIRE/FLOOD merges
Live (house features, counterfactual type and house rows)	2.41 (0.70)	19.74 (1.54)	1.0	0
No counterfactual type rows, hard type rule on	2.41 (0.90)	20.26 (1.26)	1.2	0
No counterfactual type rows, rule off	2.86 (1.04)	18.46 (1.54)	1.7	3.0
No house features	2.23 (0.72)	18.85 (1.00)	0.9	0
House features without counterfactual house rows	2.15 (0.59)	19.10 (0.90)	0.8	0

Table A.3 lists the main endpoints of the C4 service.

Table A.3: Endpoints of the C4 service

Method and path	Purpose
POST /v2/reports	Receive a report from C2 or C3 (contract c4.v2); reply with the Step 1 summary
GET /v2/reports, GET /v2/reports/<id>	List live reports; one report in full with every step
GET /v2/incidents, GET /v2/incidents/<id>	Fused incident records, newest first; one record
GET /health	Service status, loaded models and switches
GET /operator, GET /dashboard	Operator testing page; incident dashboard

Appendix B Academic Integrity and AI Use Disclosure

Generative AI models form part of the research method of this component, and their use is described in the thesis where it occurs. Table B.1 summarises it. The teacher model was used offline, during training only, and is not called by the live pipeline; the house-number reader is the only live use, as a fallback when the lexicon cannot read a number.

Table B.1: Generative AI used within the research method

Use	Model	Purpose	Verification	Sections
Teacher for the extractor (offline only)	GPT-4o mini (gpt-4o-mini-2024-07-18)	Field extraction on the training split only, distilled into mT5-small. Not part of the live pipeline: at run time the mT5-small student extracts without any API call	Six-check guard with verbatim quotes; student evaluated against gold	3.2.4, 4.4.1, 6.1.1
Drafting reference labels	Large language model (different family from the runtime extractor)	Draft gold fields and evidence-pair labels	Blind human agreement check (40 + 40 rows); systematic errors corrected in versioned datasets	3.2.3
House-number reading (fallback only)	GPT-4o mini (pinned)	Live fallback, used only when the multilingual number lexicon cannot read the house number of a '<word(s)> <road>' place; answers cached	Integer 1-99999 only; switchable; off in all reported evaluations	5.5